



西南科技大学

Southwest University of Science and Technology

学术型硕士研究生学位论文

(工学硕士)

拥塞感知的集成电路全局布线算法研究

作者姓名_____伏朝奇_____

学 号_____7020200457_____

导 师_____俞文心_____

答辩委员会成员_____贾小林（主席） 刘涌_____

_____李学俊 覃仁超 吴家菊_____

评阅人_____郭星 于啸_____

2023年5月

分类号:

学校代码: 10619

UDC:

密级:

西南科技大学硕士学位论文

(工学硕士)

拥塞感知的集成电路全局布线算法研究

伏朝奇

学科(专业)或领域名称: 软件工程

指导教师姓名、职称: 俞文心 教授

学生所属培养单位: 西南科技大学

论文答辩日期: 2023 年 5 月 25 日

A Thesis Submitted to Southwest University of
Science and Technology for the Degree of Master

(Master of Science in Engineering)

Research on Global Routing Algorithm of
Congestion-Perceived Integrated Circuit

Candidate: Zhaoqi Fu

Supervisor: Wenxin Yu

May 2023

摘 要

随着现代超大规模集成电路设计的特征尺寸不断缩小，电路元件和互连密度也不断增加，导致了频繁的布线区域拥塞问题。为了解决这一挑战，一是在布局阶段引入快速高质量的全局布线拥塞图来提高集成电路的可布线性，二是在全局布线阶段减少布线拥塞来指导详细布线生成合法解。然而，当前的全局布线算法存在以下问题：如果过于关注详细布线指导，则需要较长的运行时间；而如果过于关注布局指导，则会产生许多拥塞估计误差。

本文在调查了全局布线领域的相关理论和算法后，提出了一种拥塞感知的全局布线算法。该算法不仅能够有效降低布线拥塞，生成高质量的解来指导详细布线，还极大地提升了布线速度。这种算法能够准确估计布线拥塞情况，同时作为布线拥塞的估计工具，为可布线性布局提供指导。主要工作如下：

首先，本文对多源多目标的迷宫布线算法进行了研究，设计出两阶段的双路径多源多目标迷宫布线算法，该算法能有效降低布线搜索空间提升布线效率。随后在此基础上，本文又提出了确定性的双路径迷宫布线，在能够满足确定性要求的同时保证了布线质量，并对算法的有效性进行了证明。

其次，本文通过对弱一致性并行布线算法的研究，提出了新的网络级别的布线算法，不仅能在相同数量核心的机器上展现出更快的布线效率，更能在多核的设备中提高算法的整体收敛性能。

然后，本文提出了动态布线调度算法，对布线网络迭代不同阶段进行同步，在利用网络搜索空间相斥的布线区域时，来解决网络间冲突的布线路径，避免拥塞无法收敛。最后再与迷宫布线相结合，进一步提升布线效率，减少布线冲突。

最后，本文将网络级别布线和动态布线相结合，设计了两阶段的布线框架，该算法在第一阶段执行网络级别布线，然后通过设计的并行度下降算法来控制并行度并切换到动态布线调度算法。通过该布线框架不仅能很好地控制布线拥塞，同时还提升了布线效率。

全局布线作为组合优化问题，本文从基础的布线算法到布线的迭代算法进行了深入研究，并设计了一种拥塞感知的全局布线算法。在 ISPD-08 测试案例的实验中，本文设计的算法在布线质量上几乎与最优算法无差异，并且布线速度远远超过了所有其他布线算法。该算法取得了低拥塞高效率的效果，同时满足了指导可布线性布局 and 详细布线的需求。

关键字：超大规模集成电路；全局布线； 迷宫布线； 并行布线； 动态调度

论文类型：理论研究

ABSTRACT

As the feature size of modern large-scale integrated circuit (IC) designs continues to shrink, the circuit component and interconnect density increase, leading to frequent congestion issues in the routing regions. To address this challenge, there are two approaches: firstly, introducing a fast and high-quality global routing congestion map during the placement phase to enhance the routability of the IC, and secondly, reducing routing congestion during the global routing phase to guide the generation of legal solutions in detailed routing. However, current global routing algorithms suffer from the following issues: if they focus too much on guiding detailed routing, they require longer execution times, while if they emphasize layout guidance, they generate many congestion estimation errors.

In this paper, after investigating relevant theories and algorithms in the field of global routing, we propose a congestion-aware global routing algorithm. This algorithm not only effectively reduces routing congestion and generates high-quality solutions to guide detailed routing but also greatly improves the routing speed. This algorithm accurately estimates routing congestion and serves as a guiding tool for routability-driven layout. The main contributions are as follows:

Firstly, this paper investigates the multi-source multi-sink maze routing algorithm and designs a two-stage dual-path multi-source multi-sink maze routing algorithm, which effectively reduces the routing search space and improves routing efficiency. Subsequently, based on this, a deterministic dual-path maze routing algorithm is proposed, which satisfies deterministic requirements while ensuring routing quality, and the effectiveness of the algorithm is demonstrated.

Secondly, through the study of weakly consistent parallel routing algorithms, this paper proposes a new net-level routing algorithm that not only demonstrates faster routing efficiency on machines with the same number of cores but also improves the overall convergence performance of the algorithm in multi-core devices.

Next, a dynamic routing scheduling algorithm is proposed to synchronize the different stages of the routing network iteration, solving network conflicts in routing paths when utilizing the mutually exclusive routing regions in the network search space, thereby avoiding congestion that cannot converge. Finally, combined with the maze routing, routing efficiency is further improved, and routing conflicts are reduced.

Lastly, this paper combines net-level routing with dynamic routing and designs a two-stage routing framework. In the first stage, the algorithm performs net-level routing and then switches to the dynamic routing scheduling algorithm by controlling the parallelism using a designed parallelism reduction algorithm. This routing framework effectively controls routing congestion while improving routing efficiency.

As a combinatorial optimization problem, this paper deeply investigates from basic routing algorithms to iterative routing algorithms and designs a congestion-aware global routing algorithm. In the experiments on the ISPD-08 test cases, the proposed

algorithm achieves nearly optimal routing quality and significantly outperforms all other routing algorithms in terms of routing speed. This algorithm achieves low congestion and high efficiency, satisfying the requirements of guiding routability-driven placement and detailed routing.

KEY WORDS: VLSI; Global Routing; Maze Routing; Parallel Routing; Dynamic scheduling

TYPE OF THESIS: Theoretical Research

目 录

1 绪论.....	1
1.1 研究背景.....	1
1.2 研究现状.....	3
1.3 研究内容.....	7
1.4 本文结构安排.....	9
2 全局布线技术背景	10
2.1 全局布线问题.....	10
2.1.1 布线问题描述.....	10
2.1.2 布线问题建模.....	12
2.2 模式布线.....	13
2.3 迷宫布线.....	16
2.3.1 多源多目标迷宫布线.....	16
2.3.2 三端点迷宫布线.....	16
2.4 树拓扑结构生成.....	17
2.4.1 直线最小生成树	17
2.4.2 直线斯坦纳最小树	18
2.5 协商拥塞布线.....	20
2.6 布线迭代算法.....	21
2.6.1 拆线与重布线.....	21
2.6.2 整数线性规划布线.....	21
2.7 本章小结.....	22
3 拥塞感知的全局布线算法	23
3.1 全局布线算法流程.....	23
3.2 迷宫布线算法.....	24
3.2.1 基于协商拥塞布线的成本函数.....	25
3.2.2 两阶段的双路径多源多目标迷宫布线	25
3.2.3 确定性的双路径迷宫布线.....	28
3.3 布线迭代算法.....	29
3.3.1 网络级别的并行布线.....	29
3.3.2 动态布线调度算法.....	30
3.4 本章小结.....	35
4 实验结果与分析	36
4.1 基准测试案例.....	36
4.2 算法验证.....	39

4.3 与优秀算法对比.....	41
4.4 本章小结.....	44
5 总结与展望.....	45
5.1 总结.....	45
5.2 展望.....	46
参考文献.....	47

1 绪论

1.1 研究背景

在 20 世纪末，数字电子技术迅速发展，成为构建世界互联的基础，推动了社会生产力的进步。如今，数字电子技术已经融入到现代社会的方方面面，从智能家居、军工设备到智能手环、手机平板，再到无人驾驶汽车和航空飞机，数字电子技术已广泛应用于人们生活的各个领域。

大规模集成电路加工技术的进步则是现代数字电子技术发展的基础，集成电路是将成千上万只电子元器件组合在一起形成的电路，能够通过超大规模的集成电路加工技术来在相同面积的硅片上制造出更多数量的晶体管。随着技术的不断进步，集成度也在不断提高，在上世纪七十年代末发布的 8086 微处理器上只包含了 4 万只晶体管，而在千禧年推出的 Pentium 4 微处理器芯片的集成度上升到 4200 万只晶体管。以显卡为例在 2006 年 NVIDIA 公司推出高达 6 亿多晶体管数量的 GeForce 8800 显卡，而 2022 年发布的显卡 GTX 4090 的晶体管数量达到了 700 多亿。随着集成电路规模地指数增长以及制造工艺不断发展带来的更多设计规则约束影响，手工设计已经无法满足实际需求，因此有了 EDA（Electronic Design Automation，电子设计自动化）的产生。

随着集成电路规模地指数增长以及制造工艺不断发展带来的更多设计规则约束影响，手工设计已经无法满足实际需求。而且集成电路设计公司在设计的芯片在进行制造时也需要面对芯片设计对良品率的影响，而设计差的芯片更容易导致制造失败，同时流片所需要的费用却十分高昂，有不少的集成电路设计公司就是因为制造流片的问题而消失在历史中。EDA 工具作为芯片辅助设计工具，极度地提升了芯片设计的效率，使得相同时间内能将设计迭代更多次数，从而同时提高了芯片的设计质量。所以说，集成电路的规模和工艺的进步促进了 EDA 工具的诞生，而 EDA 工具的诞生反过来进一步促进了集成电路设计的发展。

物理后端设计在 EDA 的流程中，是将逻辑电路映射到物理电路的过程，它可以在保证电路正确性和可靠性的同时提高性能和降低功耗，同时也可以加快设计的速度和准确性。专用数字集成电路（ASIC）物理后端设计流程如图 1-1 所示，前端设计人员使用硬件描述语言设计出芯片功能生成文件，然后就可以作为物理设计的输入。各个部分功能如下：

1) 逻辑综合：加载库和设计文件、施加时序约束和设计规则约束条件，然后通过翻译、逻辑门到物理门映射和逻辑优化来生成对应的门级网表文件，最后对生成结果进行分析并写出网表文件。

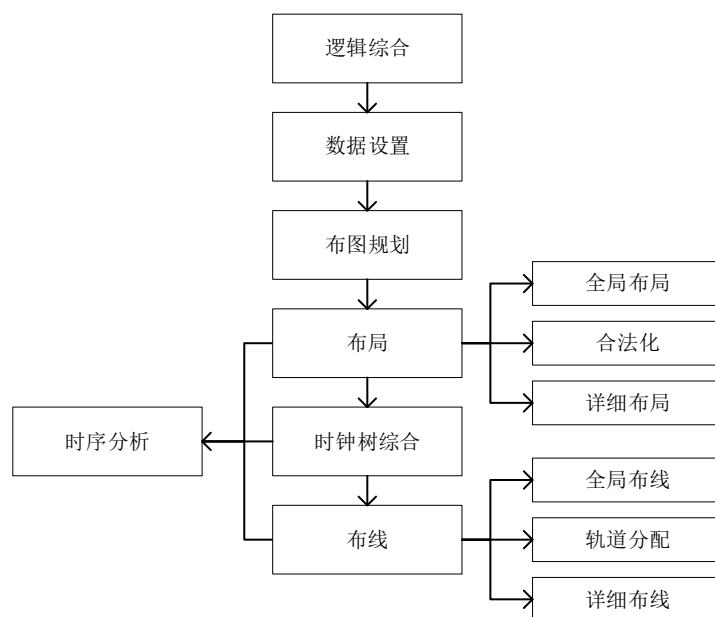


图 1-1 ASIC 物理后端设计流程

2) 数据设置：加载必要的综合数据、逻辑库、约束和网表以及物理设计数据、物理库、工艺文件和电容电阻寄生参数模型文件，创建设计库和初始化设计单元，应用时序和优化控制帮助优化性能、功耗和芯片面积，最后执行库文件、电容电阻寄生参数模型、时序和约束检查。

3) 布图规划：确定芯片边缘大小、核心大小和布局行高，以及输入输出块、电源块和填充块的位置，随后确定宏的物理位置和标准单元布局约束，最后设置芯片的供电电源网。

4) 布局：全局布局确定布局单元的大致位置，而合法化则会把单元与布局行列进行对齐，详细布线会进行一些优化。根据布局的目标不同，也会分为线长驱动的布局（减少总线长）、拥塞驱动的布局（减少拥塞）、和时序驱动的布局（减少时序路径违例）。

5) 时钟树综合：主要是把时钟源与对应的触发器相连，然后降低时钟源到不同触发器的最大延迟、不同触发器之间的最大延迟差和时序电路功耗。

6) 布线：全局布线将跨越网格的网络进行连接生成指导（guide）文件，轨道分配根据 guide 选择轨道对网络进行实际走线，详细布线会根据设计规则约束来修改不满足要求的连线。而布线根据目标不同，也分为经典布线（减少拥塞）和时序驱动的布线（减少时序路径违例）。

7) 时序分析: 时序分析几乎贯穿物理设计流程所有位置, 在每个部分都可以进行时序分析来获取电路的时序情况, 并根据违例的路径对门的位置大小和网络连接等进行优化。

布线则是物理后端设计中不可或缺的一环, 随着现代超大规模集成电路设计的特征尺寸不断缩小, 片上通信变得非常复杂。电路元件和互连密度的增加给现代 VLSI 布线器带来了巨大的挑战。因为不同网络间连接的数量增大和布线拥塞的集中, 同时在芯片上的布线资源是有限的, 导致当今的设计出的芯片更难以布线。超大规模的集成电路带来的大量的布线需求, 直接根据布线需求和设计规则进行布线是不符合实际的。为此, 布线被分成了全局布线和详细布线。全局布线在粗化之后的网格中进行布线, 生成一系列粗化的网格布线数据。详细布线再使用数据来进行实际的连线。尽管全局布线忽略了轨迹和设计规则检查等布线细节, 但它生成的互连信息非常接近最终的布线实现。它可用于准确估计互连拓扑、线路长度、拥塞和时序。除了可布线性问题, 特征尺寸的不断缩小给制造过程带来了很大的困难。布线是设计过程中考虑设计制造产量的关键步骤。它将决定一个布局是否有高产量。通孔是电路故障的一个主要来源, 其工艺变化较大, 以一种不太可预测的方式影响电路的时序成品率。因此, 线长和通孔最小化是全局布线的另一个目标。

全局布线会根据输入文件中的参数把布线的区域划分成对应的矩形网格单元, 并使用粗粒度的三维布线图表示。在这个三维布线图中, 通过给边和顶点分配容量和各种约束条件, 使得整个布线拓扑和分层分配能够在考虑布线可能性的情况下得到优化。全局布线问题作为非确定性多项式问题、电子设计自动化中的一环, 如何降低布线拥塞、缩短布线时间, 以此减少芯片设计周期、提升芯片性能从来没有被学术界与工业界忽视过。

此外, 国际上对全局布线问题的研究在 2007 年左右达到高峰, 在随后的十年中逐渐减缓, 但仍有相关的间歇性研究产生。然而, 在国内的研究方面, 如果在知网上搜索“全局布线”, 则可以发现最近的相关文章是 2013 年发表的粒子群优化的全局布线^[1]。因此, 增加全局布线问题的中文资料也具有重要的现实意义。

1.2 研究现状

全局布线算法在这么多年的研究和发展下, 主要分为树拓扑生成、网络分解、布线迭代三个过程, 如图 1-2 所示。

布线需要将芯片单元的每一个网络的引脚连接到一起, 每个网络的引脚连接时则会形成一个树结构。因此在全局布线中, 会根据每个网络的各个引脚的

位置先生成是拓扑结构。目前主要有直线斯坦纳最小树与直线最小生成树两种，直线最小生成树相比斯坦纳树有较强的灵活性，而斯坦纳最小树（主要通过 FLUTE^[2]查表）生成速度更快更精确。此外，段移动技术会与树生成相结合，它能识别出树拓扑结构中能够移动而不产生现场变化的段，然后将拥塞区域的段向非拥塞区域移动一旦树拓扑结构固定，无论是模式布线还是迷宫布线，都不能在水平和垂直方向之间转移布线资源需求，因此 FastRoute^[3]还提出了拥塞驱动、通孔感知的树生成算法，依据布线区域的拥塞图映射出新的引脚位置并用 FLUTE 算法生成直线斯坦纳树，鼓励在拥塞方向和拥塞区域使用布线需求较少的拓扑结构，而算法^[4]提出使用过滤斯坦纳节点的方式来构造斯坦纳树，Latha^[5]等人对大网络的斯坦纳最小树进行了线长和内存的优化。SALT^[6]算法在斯坦纳最小树和斯坦纳树之间做了平滑处理以实现更低的时间复杂度和延迟。

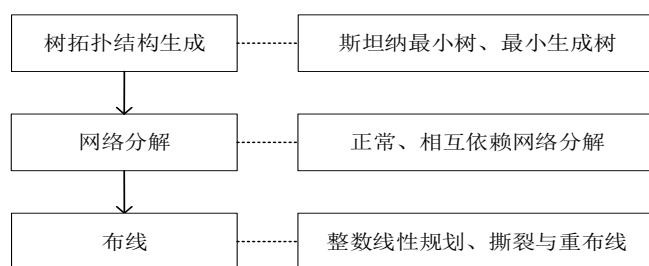


图 1-2 全局布线流程

由于布线中迷宫布线时同时只能对一个网络的一条边进行布线，所以会将网络分解成单独的边。主流方法则是把相邻两个节点（包括引脚和斯坦纳节点）之间的线作为一条边存储，而 MAIZEROUTER^[7]则对网络拓扑结构显示编码，模拟发生在布线段之间的全局交互，采用了相互依赖的网络分解，将每个网络连线的直线集合进行存储，在每个分段的基础上操作，只有那些受局部变化影响的边将被修改，以适应候选路径选择的极端边缘移位，允许对这个底层结构进行低成本的增量更新。

在布线中，形成了整数线性规划（integer linear programming(ILP)）和拆线与重布线（rip-up and reroute (R&R)）两种主要基础迭代方案。

在整数线性规划算法中，随着布线网络数与网络候选路径的增加，布线速度会愈加缓慢，而布线的总网络数远远超过整数线性规划能承受的范围，解质量也受到候选路径的影响，但是能够生成当前输入下的最优解。因此如何挑选 ILP 的网络与网络的候选路径尤为重要。Albrecht^[8]等人提出了多商品流的近似算法，通过使用线性规划松弛来解决全局布线问题。BoxRouter^[10]提出了一种渐进式的整数线性规划算法，该算法在拥塞最严重的区域生成一个包围盒，对于范围内的网络执行整数线性规划，随后不断地扩展包围盒的范围，对新的网络

进行布线，以此削减每次执行的网络数量避免计算爆炸。CGRP^[11]则是根据拥塞程度生成了一系列关键区域，从那些区域中成比例选出 5000 个关键边（拥塞高），然后从中选出 1000 个网络并为每个网络选择 10 条候选路径作为输入，最后执行 ILP 求解。LCGRP^[12]则是在 CGRP 的基础上引入了局部拥塞网络的考量，并对网格划分进行偏移来降低拥塞。Fontana [13]等人则使用了 ILP-GRC 同时进行单元移动和网络布线。

拆线与重布线的使用更为普遍，也会被穿插在很多整数线性规划算法中来生成候选路径。R&R 布线框架从初始化生成的布线结果开始，迭代地提高布线质量。在拥塞集中位置的网络会优先拆线，然后在拥塞更小的区域重新布线以提高总体的可布线性。NCTU-GR 2.0^[14]、NTHU-Route 2.0^[15]、FastRoute 4.0^[26]、CUGR^[16]等大部分布线器均采用该方案作为基础算法，而 R&R 当前也是应用最广泛的方案。

两种方案中，无论哪种都需要算法来生成一条路径。而生成路径的算法主要包括模式布线和迷宫布线，通过模式布线能够快速生成路径来作为初始化使用，而迷宫布线则是具有强大的路径搜索能力，是主要的路径生成手段。

模式布线^[25]使用预定义的模式来布线 2 引脚网，最常用的是 L 型、Z 型和 U 型。模式布线应用于二维布线，因此 CUGR^[16]提出了三维的模式布线，传统的模式布线只生成二维拓扑，而三维模式布线直接生成三维拓扑，不需要额外的层分配阶段。以 L 形布线为例，传统的模式布线在每个双引脚网络中只能选择两条可能的路径，而且有些路径即使在二维网格图中没有引起任何溢出，也可能在分层后无法进行布线。然而，提出的三维模式布线结合了二维模式布线和层分配。在每个双引脚网络的 $2 \times L \times L$ 可能选项中选择一条路径，其中 L 是层数，使用动态规划可以在线长和拥塞方面保证找到最优的解决方案。由于模式布线限制了布线路径形状的模式，因此可以加快全局布线过程。因此，通常采用模式布线来布线大部分网络以节省运行时间，而并行的模式布线^[9]被提出进一步提升布线效率。在快速布线中，每一个斯坦纳树被分解成两个引脚网络之后，使用 Z 型模式布线来布线每一个双引脚网络。虽然模式布线可以加快布线过程，但其质量可能比迷宫布线差得多。迷宫布线器确保找到代价最小的布线，但模式布线只考虑可能布线的一小部分。对于跨 $m \times n$ 网格的双引脚网络，L 型模式布线只考虑 2 条不同的路径，Z 型模式布线只考虑 $m \times n$ 条不同的路径。因此，在许多情况下，模式布线无法找到良好的布线路径来避免拥塞。FastRoute 2.0^[17]提出了基于动态规划的单调布线来在迷宫布线和模式布线找到一个平衡点，在迭代前期来快速解决一部分拥塞生成路径。FastRoute 4.0^[26]随后提出由两个直角引脚连线组成的 3 拐点布线，最多可以形成三个拐点。在解决拥塞问题上，相比 L/Z/U 型布线三拐点布线具有更大的灵活性，但是时间复

杂度更高, 达到了 $O(M*N)$ 。不过与单调路线^[17]和迷宫布线相比, 3 拐点布线具有通孔更少的优点。尽管单调布线能够有效地识别无绕线路径, 但当需要绕线路径来避开障碍物或拥挤区域时, 单调布线无法取代迷宫布线。绕道被视为远离目标。Liu W H 等人^[18]提出了一种极其快速的布线算法, 称为单向单调布线算法, 该算法能够在与单调布线算法相同的时间复杂度下寻找绕道路径并运行。单边单调布线使用最小的水平或垂直距离在有限的区域内识别最小代价的布线路径。与迷宫布线相比, 单向单调布线仍然为解决溢出问题提供了有限的解决空间。随后在单向单调布线的基础上提出了一种混合单向单调(HUM)布线算法^[18], 用于寻找比单向单调布线提供的更大的解空间。HUM 布线概念假设 B 中的每个节点都是连接起始节点和目标节点的中间点, 最终路径由两条路径组成, 即连接起始节点和中间点的路径和连接中间点和目标节点的路径。

在迷宫布线算法上, NCTU-GR 2.0^[14]在有界长度迷宫布线 (bounded-length maze routing, BLMR) 上提出最优 BLMR 及启发式 BLMR 演算法来解决有界长度迷宫布线问题。虽然提出的启发式 BLMR 方法不能保证最优解, 但它比最优 BLMR 方法快得多。最优 BLMR 和启发式 BLMR 的不同之处在于启发式 BLMR 只保留从源到当前节点的一条路径, 其他路径被丢弃, 因此启发式 BLMR 的主要问题是确定哪个路径候选应该被保留。NCTU-GR 2.0 随即开发了一种启发式碰撞预防方法, 使每个线程能够通知其他线程, 如果某些网格边已经被当前布线的网络在前一次迭代中使用。因此, 每个线程都可以估计使用这些网格边的风险。更多细节, 每个线程在当前迭代开始之前在拥塞图上标记当前布线网络使用的网格边缘, 并在回溯结束后清除它们的传播数据。标记的网格边缘分配额外的布线成本。当线程传播到标记的网格边缘时, 标记的网格边缘的额外布线开销鼓励线程绕过标记的网格边缘以防止冲突。

MGR^[19]利用模式布线和层分配提供了初始解, 使粗化阶段可以根据布线资源实际使用情况计算出较高层网格图的布线能力, 然后对布线区域进行粗化, 过程中生成较小尺寸的网格图, 用于三维迷宫布线。随后使用网络流算法将布线解在行上传播到较细的下层网格结构中, 为了减少布线通孔数放松问题允许每个高级网络选择新的布线层, 而不是金属层决定在其高层解决方案。这种放松有助于 MGR 纠正任何层分配错误在上一级。将粗化问题分解为片上网络流的子问题, 该分解解决了复杂的非粗化问题, 但同时也考虑了使用单一切片的所有网络之间的竞争。

在初始布线之后, 那些有违规行为的网络将被撕裂, 并可能通过迷宫布线进行多次撕裂和重布线迭代。然而, 由于搜索空间较大, 在整个三维网格图上进行迷宫布线将非常耗时。CUGR^[16]提出了一种多级三维迷宫布线算法, 以实现全图搜索, 同时在时间和布线质量之间进行协调。所提出的迷宫路径规划技

术分为迷宫路径规划和细粒度迷宫路径规划两个层次，每个层次都有不同的用途。迷宫路径规划的目标是寻找一个较小但高可布线性的搜索空间，而细粒度迷宫路径规划的目标是在搜索空间内寻找一条实际开销最小的路径。同时并行布线技术^[48]被应用到加速三维布线。SPRoute^[40]通过探索网络级别的并行和细粒度的并行来实现迷宫布线阶段的加速。网络级并行的实现。全部线程共同使用一个全局网格，该网格包含网格边缘上的使用信息。线程在拆卸过程中自动减去使用量，或者在建立布线时添加使用量。每个线程都有一个本地网格，该网格存储网格节点上的所有信息，包括它与源节点的距离、扩展边界的父节点等。在细粒度并行中，网格图上的节点在工作列表中时被称为激活态，并行循环从工作列表中的一组源节点开始。在展开过程中，每个线程首先从工作列表中弹出一个节点，并为自己及其四个邻居节点获取锁。然后松弛节点的边缘，计算新的邻居的距离。

全局布线是三维布线，由此也产生了二维布线与层分配^{[46][47]}相结合和三维直接布线两种方式。BoxRouter 2.0^[20]、DpRouter^[21]和NTHU-Route^[23]采用的一种常见的三维全局布线策略，首先将三维网格图压缩成二维网格图，将三维资源映射到二维上，然后执行二维全局布线。然后将得到的无溢出的二维全局布线解逐层投影到三维空间中。这种策略已经被许多高性能布线器证明在平衡运行时间和布线质量方面很好。在所有布线完成后，通过动态规划或者增强的启发式^[22]的层分配算法，将二维路径的路段分配到不同的层。另一个不太常见但功能强大的策略是在三维网格图中直接布线网络，例如，FGR^[24]使用基于离散拉格朗日乘子的代价方案直接在三维布线空间中应用迷宫布线，MGR^[19]采用多层技术将网格图粗化在较小的空间内完成三维迷宫布线，CUGR^[16]基于模式布线与层分配相结合和多级三维迷宫布线求解。

设计一种低拥塞高效率的全局布线算法，能够在布局阶段进行快速高质量的拥塞估计，并生成低拥塞的解来指导详细布线生成合法解，是当前的研究难题。当前已有研究关注于拥塞驱动的全局布线，但存在两个问题：过于关注详细布线指导会导致长的运行时间，过于关注布局指导会引发拥塞估计误差。因此，如何克服这些问题，实现低拥塞高效率的全局布线算法，仍然具有挑战性。

1.3 研究内容

在拥塞感知的全局布线中，成百上千万的网络需要在很小的矩形区域内有限的布线资源下连接，连线都需要占用线宽并满足线间距要求。因此不仅会面临网络过多而导致的网络冲突、拥塞严重的问题，也需要面临网络线长、通孔

过多导致芯片性能、可制造性变差的问题，以及布线时间过长而增加设计周期。因此如何研究出能降低布线拥塞、减少布线的线长与通孔数量以及提升全局布线效率的算法来提升可布线性布局的速度和质量以及指导详细布线完成高质量的布线具有重要的意义。

针对上面的布线问题，本文对于下面三个布线方面的问题展开了研究：多源多目标的迷宫布线算法、撕裂与重布线迭代算法框架、并行布线和网络冲突解决算法。

在全局布线算法研究中，都会不同程度地使用迷宫布线算法来在更大的解空间中寻找解，因此对于多源多目标的迷宫布线算法优化在布线中具有重要的价值。在全局布线中，存在撕裂重布线和整数线性规划布线两种布线迭代算法框架，设计出合适的迭代算法框架不仅能提升布线质量也能提升布线效率。由于全局布线算法发展目前没有重大的突破，所以当前很多的工作也落到了如何设计出高质量的并行算法在不影响布线拥塞的情况下来提升布线效率，对各个网络进行布线，网络间的路径都会存在不同程度的交叉而互相影响布线质量，因此需要合适的布线算法来调度布线网络，解决网络间的布线冲突。

对此本文做出了以下研究工作：

- (1) 对多源多目标的迷宫布线算法进行了研究，从而设计了两阶段的双路径迷宫布线算法来降低布线搜索空间提升布线效率。虽然在快速布线拥塞估计具有较大的意义，但是该算法具有不确定性，不利于程序调试也不适合工业应用。于是在此基础上，本文又提出了确定性的双路径迷宫布线，在能够满足确定性要求的同时保证了布线质量，在算法描述之后证明算法的有效性。
- (2) 通过对弱一致性并行布线算法的研究，提出了新的网络级别的布线算法，不仅能在相同数量核心的机器上展现出更快的布线效率，更能在多核的设备中提高算法的整体收敛性能。
- (3) 提出了动态布线调度算法，对布线网络迭代不同阶段进行同步，序列化撕裂网络阶段，并对布线和网络拓扑结构更新部分进行了并行化，在利用网络搜索空间相斥的布线区域时，来解决网络间冲突的布线路径，避免拥塞无法收敛。最后再与迷宫布线相结合，进一步提升布线效率，减少布线冲突。
- (4) 将网络级别布线和动态布线相结合，设计了两阶段的布线框架，该算法在第一阶段能对布线资源充足区域的网络进行快速布线，然后通过设计的并行度下降算法来控制并行度，在满足指定条件时切换到第二阶段执行动态布线调度算法，来对布线拥塞区域的网络进行布线。通过该布线框架不仅能很好地控制布线拥塞，同时还提升了

布线效率。

1.4 本文结构安排

本文分为五章，第一章首先介绍了电子设计自动化（EDA）的产生背景以及专用数字集成电路设计中后端物理设计的主要流程。重点说明了布线在其中的角色，并介绍了拥塞感知的全局布线的国内外现状。最后对本文的工作进行了总结。

第二章描述了拥塞驱动的全局布线所面临的问题，并对全局布线问题进行了建模。详细阐述了全局布线中的溢出和线长等衡量指标，并介绍了模式布线和树拓扑结构生成算法作为布线初始化算法的工作原理。然而，这两种算法无法解决布线拥塞问题，因此本章还介绍了迷宫布线和布线迭代等解决拥塞问题的算法。

第三章首先介绍了拥塞感知的全局布线算法的总体流程，然后讨论了当前迷宫布线算法存在的问题。接着详细描述了本文提出的迷宫布线算法，并对算法的有效性进行了证明。最后详细阐述了本文提出的两阶段的布线迭代算法。

第四章先介绍了实验环境、使用的基准测试案例以及测试案例的文件格式。然后通过图表展示了提出的算法的效果，并将其与在测试基准上表现最好的开源全局布线算法进行了比较，比较指标包括拥塞溢出、总线长和布线迭代时间。最后对实验结果进行了总结。

第五章对本文的工作内容进行了总结，并梳理了未来的工作方向。

2 全局布线技术背景

本章主要对全局布线相关技术背景进行了描述，首先详细阐述了布局结果到全局布线问题产生的过程，随后对全局布线问题进行了建模，最后对当前的主要全局布线算法进行了介绍。

2.1 全局布线问题

2.1.1 布线问题描述

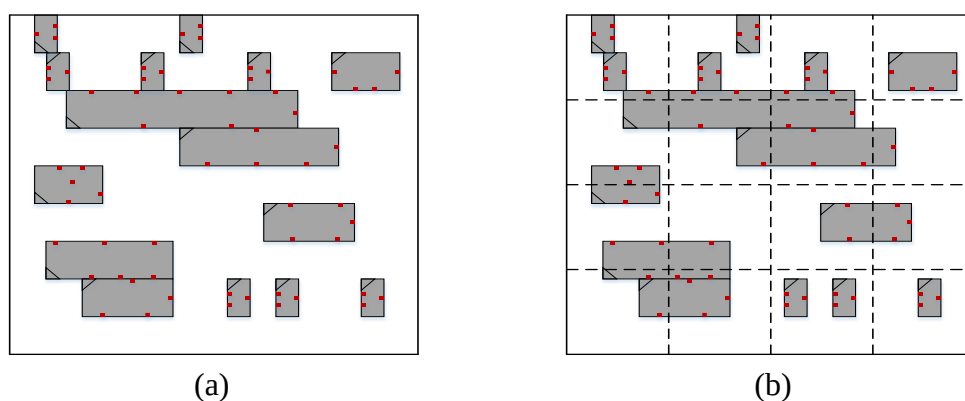


图 2-1 全局布线区域划分

全局布线时期，芯片上的标准单元和宏单元都已经确定好实际位置。如图 2-1 (a)所示，灰色矩形为具有与、或、非或者更复杂功能的单元，单元上的红色矩形则是输入或输出引脚，具有复杂功能的芯片就是由无数单元（标准单元、宏以及触发器等）连接上后构成的。某一个单元的引脚输出会作为一个或多个单元引脚的输入，而这些引脚则组成了一个网络。如图 2-2 所示，在图 (a) 中，有两个双引脚组成的网络，而图 (b) 中有一个三引脚和双引脚的网络，每个红色矩形中就包含了一个网络。

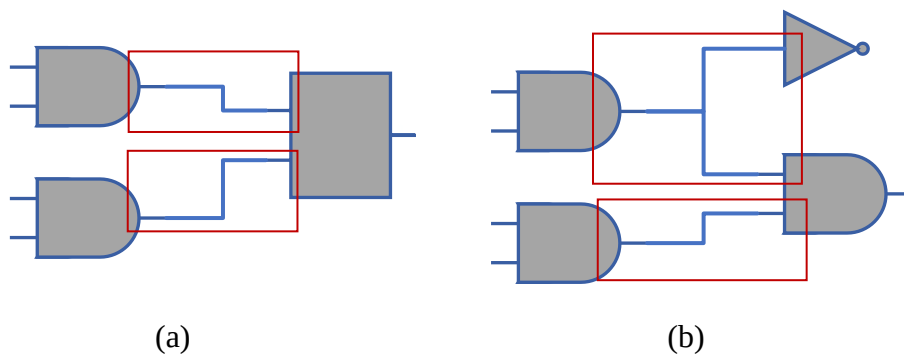


图 2-2 布线网络

全局布线则需要根据约束条件将所有网络进行粗连接，如图 2-1 (b)所示，这里布线区域被切分成了 4 乘 4 的网格区域，而单元的引脚也被分配到了不同的网格中。此时，全局布线在连接引脚时，只需要把引脚所属的网格进行连接。如图 2-3 (a)所示，图中包含了两个需要全局布线的网络，分别是三引脚的网络和双引脚网络。对于三引脚网络实际布线中需要连接三个引脚，但是全局布线是以网格为基础的粗布线，因此实际需要连接的网络如图 2-3 (b)所示。由于右上网格中的两个引脚属于一个网络，两个引脚则被合成了同一个点，因此转换成了图 (b)所示的两个双引脚的布线网络，此时需要连接的不是单元引脚，而是布线网络。

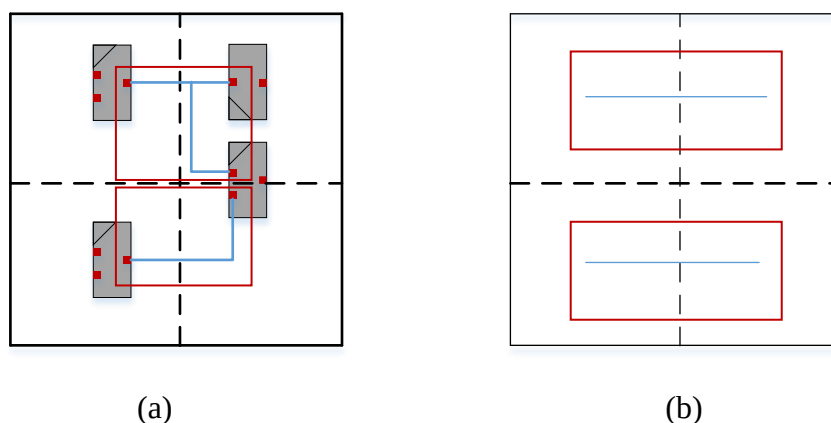


图 2-3 全局布线网络

全局布线中，单元网络被转换成了网格网络，而且实际中金属连线是需要占用物理空间资源的，因此相邻两个网格之间的金属线数量是有限制的，具体数量是根据金属线的宽度、金属线之间的间距以及网格的宽度得到的。如图 2-4 (a)所示，相邻网格边的长度为 6，若金属线宽和线间距均为 1，则能够连接的金属线数量为 $6/(1+1)=3$ 。而布线区域可能会存在障碍（无法布线的区域），障碍会降低相邻网格间的金属连线数量。如图 2-4 (b)所示。黑色矩形为障碍物，该区域金属线无法通过，上下的布线区域分别被占用了 2 和 4 单位，因此金属线数量上限分别为 2 和 1。

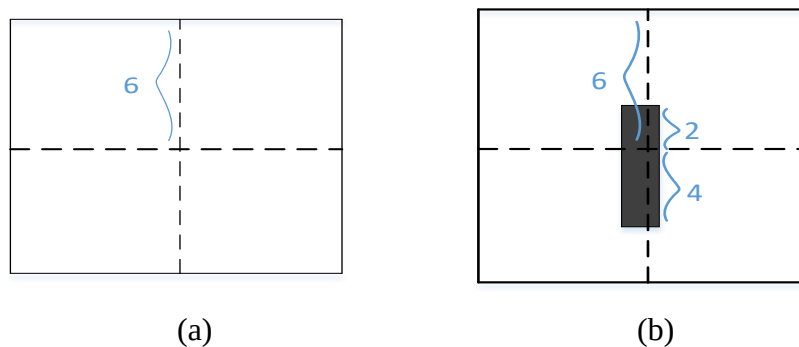


图 2-4 网格边容量

在芯片制造上，由于单元规模地增大，没有足够的布线资源连接单元，因此产生了多层的布线资源。单元会被放置在第一层，而布线除了在第一层外，还有额外数量的金属层可以布线，不过正常情况下，某一层只能进行横向或者竖向的连线，而层与层之间的金属线通过通孔进行连接。如图 2-5 所示，M1 只能横向布线，而 M2 只能竖向布线，因此连接两个红色引脚时，需要通过金属线→通孔→金属线→通孔的方式连线，而不同层的金属线宽和线间距不同也会导致不同层的布线资源不同。

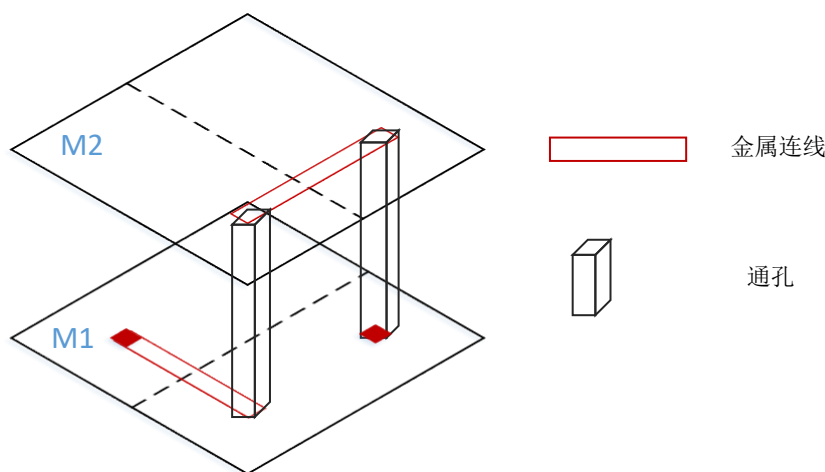


图 2-5 全局布线网络路径

2.1.2 布线问题建模

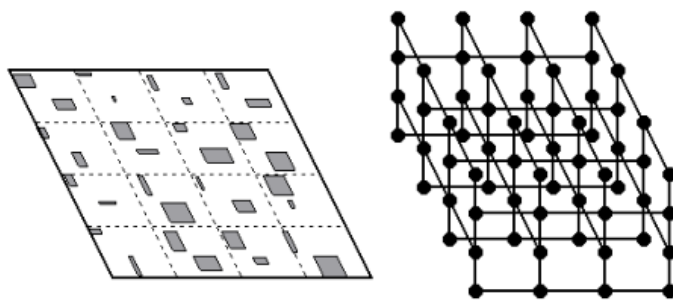


图 2-6 布线区域分解为布线网络

在全局布线中，布线区域被划分为矩形网格单元，并使用粗粒度的三维布线图进行表示，在这个三维布线图中，单元的位置都固定在最底层，最底层上面有数量不定的金属层，不同层的单位长度的容量不同，而且某些位置存在阻碍，导致一部分路径被占用，使得某些网格边的容量减少。如图 2-6 所示，每个金属层均匀地划分为相同的网格，全局布线问题可以建模为网格图 $G(V, E, N)$ ，其中 V 表示网格单元 $\{g_1, g_2, g_3 \dots\}$ 的集合， E 表示相邻（包括上下层的通孔

边) 网格之间边 $\{e_1, e_2, e_3 \dots\}$ 的集合, N 是布线网络 $\{n_1, n_2, n_3 \dots\}$ 的集合, p_n 则是网络 n 的连线路径。每个网格边 e 有它的最大布线容量表示为 $c(e)$, 每条通过边 e 的网络布线根据宽度和间隔规则消耗一定的布线资源 $d(e)$, 通过边 e 的当布线需求 $d(e)$ 超过 e 的布线容量时, 该网格边 e 就会产生溢出, 也就是拥塞。

(1) 溢出

公式 2-1 是边 e 的溢出计算方法, 总溢出计算如公式 2-2 所示。

$$overflow(e) = \begin{cases} d(e) - c(e), & d(e) > c(e) \\ 0, & d(e) \leq c(e) \end{cases} \quad (2-1)$$

$$TOF_{overflow} = \sum_{e \in E} overflow(e) \quad (2-2)$$

(2) 线长

公式 2-4 计算总线长, 公式 2-3 中的 $p_n \cap e = 1$ 表示网络 n 的布线路径 p_n 经过网格边 e 。

$$wl(n) = \sum_{e \in E} p_n \cap e \quad (2-3)$$

$$TOF_{wl} = \sum_{n \in N} wl(n) \quad (2-4)$$

在评估能同时指导布局和详细布线的全局布线算法的解决方案质量时, 应优先考虑其控制拥塞溢出的能力, 其次是运行时间, 而线长在拥塞估计中应作为最后考虑的因素。

2.2 模式布线

(1) L、Z、U 型布线

给定一组双引脚网络, 全局布线器需要在对应的容量约束下找到每个网络的路径。大多数网络都是通过最短路径搜索来确定的, 迷宫布线算法如迪杰斯特算法和 A*搜索可以保证获取到两个引脚之间的最短路径。但是, 在全局布线初始阶段不一定需要过于精准的布线路径, 而这些方法会占用大量的布线时间, 特别是在生成的拓扑结构是由单个引脚到引脚的连接组成时, 如 L 型布线网络。在实际布线中, 存在许多网络物理距离近路径短的网络, 而且弯道路径也很少。因此, 通常会在树拓扑结构生成后使用模式布线来进行初始化布线。模式布线使用一个预先定义好的模式布线双引脚网络, 这使得引脚到引脚的连接的形状成为固定的。模式布线的拓扑结构主要包括下面三种。给定一个 $m * n$ 大小的包围盒中, 有两个引脚 $(x1, y1), (x2, y2)$ 。

L 型如图 2-7 (a) 所示, 存在一个拐点, 布线方式只有 $(x1, y1) \rightarrow (x1, y2) \rightarrow (x2, y2)$ 和 $(x1, y1) \rightarrow (x2, y1) \rightarrow (x2, y2)$ 两种。

Z型如图 2-7 (b) 所示, 存在两个拐点, 布线方式则有 $(x1, y1) \rightarrow (x1, y) \rightarrow (x2, y) \rightarrow (x2, y2)$ $y \in (y1, y2)$ 和 $(x1, y1) \rightarrow (x, y1) \rightarrow (x, y2) \rightarrow (x2, y2)$ $x \in (x1, x2)$ 两类, 总共有 $x2 - x1 + y2 - y1$ 种布线路径。

U型如图 2-7 (c) 所示, 与 Z 型不同的是, U 型的范围从两个引脚坐标扩展成了 $m * n$ 的边界包围盒, 所以有 $m + n$ 种布线路径。

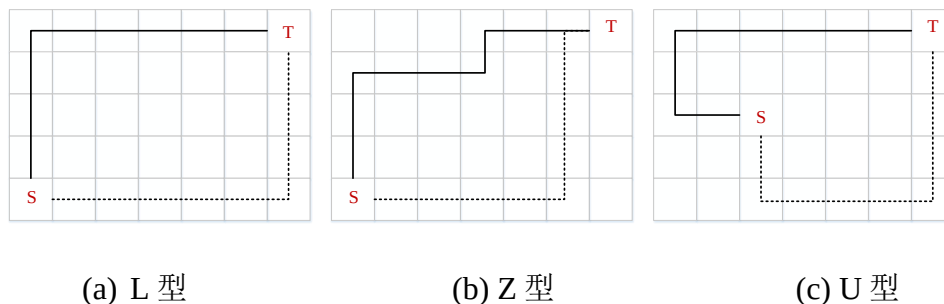


图 2-7 三种模式布线

(2) 3 拐点布线

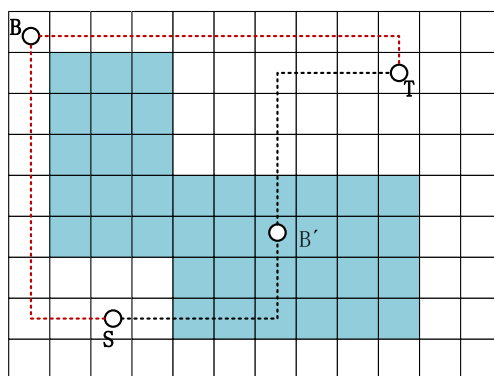


图 2-8 三拐点布线

由 FastRoute 4.0^[26]提出的 3 拐点布线由两个直角引脚连线组成, 最多可以形成三个拐点。在解决拥塞问题上, 相比 L/Z/U 型布线三拐点布线具有更大的灵活性, 但是时间复杂度更高, 达到了 $O(M*N)$ 。不过与单调路线^[17]和迷宫布线相比, 3 拐点布线具有通孔更少的优点。图 2-8 显示了两个引脚之间两种可能的 3 拐点路线, $S \rightarrow B \rightarrow T$ 和 $S \rightarrow B' \rightarrow T$ 。在这个案例中, L/Z/U 型布线都无法避免标记为阴影的拥挤区域, 而 3 拐点路线 $S \rightarrow B \rightarrow T$ 可以以尽可能少的拐点实现无拥塞布线。

(3) 单调布线

模式布线使用预先定义好的模式来布线 2 引脚网络。由于模式布线限制了布线路径形状的模式, 因此可以加快全局布线过程。因此, 通常采用模式布线来布线大部分网络以节省运行时间。在快速布线中, 每一个斯坦纳树被分解成两个引脚网络之后, 使用 Z 形的模式布线来布线每一个两个引脚网。虽然模式

布线可以加快布线过程，但其质量可能比迷宫布线差得多。在许多情况下，模式布线无法找到良好的布线路径来避免拥塞。有效的方法是在迷宫布线和模式布线之间找到一个平衡点，这样可以使质量比模式布线更好，但是运行时间更接近于这个平衡点。

单调布线^[17]其基本思想是寻找 2 引脚网络的最佳单调布线路径。设一个引脚为源 S，另一个引脚为目标 T。从 S 到 T 的单调布线路径是布线网格上从 S 到 T 的一条路径，它总是指向 T。图 2-9 显示了从 S 到 T 的两条不同的单调布线路径，但是所有的单调布线路径都不会走出 S 和 T 的边界盒。从 $m \times n$ 网格的一个角到对角的单调布线路径总数为 $\frac{(m+n-2)!}{(m-1)!(n-1)!}$ 。单调布线路径的一个重要性质是，对于 S 和 T 的边界盒中的每一个网格点，只有一个或两个网格点可以作为从 S 到它的任何单调布线路径的前身。

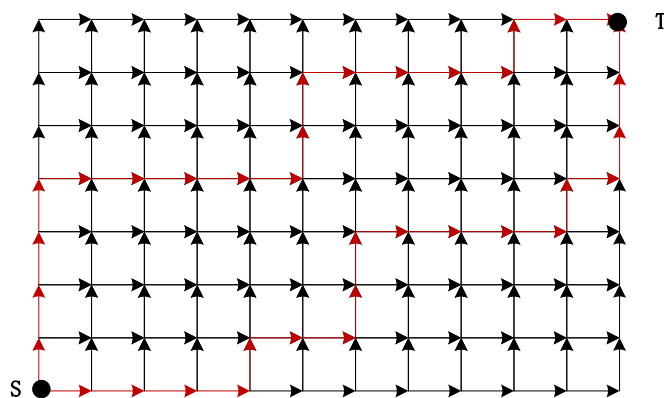


图 2-9 单调布线

由于单调布线只会朝着固定的方向走线，而不会通过回退来绕过拥塞区域，Liu 提出了图 2-10 所示的单调算法^[18]，该算法能够在与单调布线算法相同的时间复杂度下寻找绕道路径并运行。单边单调布线使用最小的水平或垂直距离在有限的区域内识别最小代价的布线路径。与迷宫布线相比，单向单调布线仍然为解决溢出问题提供了有限的解决空间。随后在单向单调布线的基础上提出了一种混合单向单调(HUM)布线算法^[18]，用于寻找比单向单调布线提供的更大的解空间。

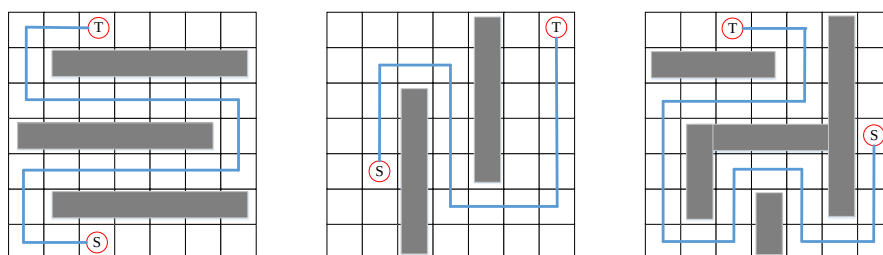


图 2-10 混合单调布线

2.3 迷宫布线

在全局布线中网络分解后，需要对分解出的两个引脚对应的网络进行连接，而搜寻网格间路径的算法是 Lee 算法^[27]。该算法也叫做迷宫布线算法，迷宫布线算法一直被认为是蛮力算法，因为它允许所有可能的路径。对于任意代价函数，迷宫布线使用最短路径连接源点和目标点。简单的迷宫布线实现通常使用广度优先搜索或迪科斯彻算法，然而使用 A* 进行优化后的迷宫布线算法^[28]会有更好的速度提升。

2.3.1 多源多目标迷宫布线

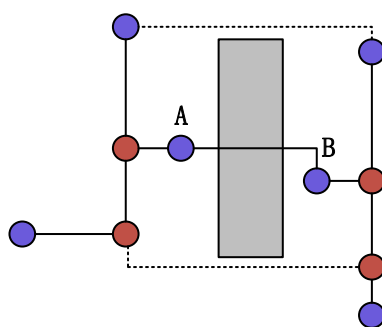


图 2-11 多源多目标迷宫布线

多源多目标迷宫布线^[17]是对基于 A* 的迷宫布线算法的优化，原有的迷宫布线算法只会寻找两个引脚之间的路径，而此算法扩展了撕裂网络的两个引脚对应的搜索集合，因此能获取到更大的解空间。图 2-11 展示了包含 6 个引脚网络，该网络被划分为两个子树。在使用传统的迷宫布线算法时，无论搜索空间有多大，只能得到从 A 开始到 B 结束的路径。然而，其他可能的路径连接两个子树也可能解决方案来实现这种多引脚布线。为了克服传统迷宫布线的缺点，多源多目标布线把图 2-11 中左侧的 3 个引脚对应的网格作为起点集合，把右边的 3 个引脚对应网格的集合作为目标集合。这样，布线器可以获得一条更好的路径，不仅避免了阻塞区域，而且获得了更短的线长。

2.3.2 三端点迷宫布线

如图 2-12 所示，不同于普通迷宫布线，三端点迷宫布线^[19]会把含有多引脚的网络分解成含有 3 个端点的网络，然后将 3 端点网络进行分解，最后对网络进行布线。三端点迷宫布线通过分别从 3 个端点向前传播，把 3 个传播的代价加在一起，然后把和最小的点设置为斯坦纳中间节点。从斯坦纳节点到端点的传播的反向回溯可以得到最优解。

因此，针对三引脚或三引脚以上网络的三端迷宫布线算法如下。三端迷宫布线算法不是撕开两个引脚的边缘，而是撕开连接到共享斯坦纳节点的三个两个引脚的边缘，生成 3 个独立的等待连接的端点。然后开始迷宫布线传播的每个端点，直到它满足任何一个其他两个端点。通过检查三个传播过程所访问的区域，找到了代价最小的斯坦纳节点。该算法利用斯坦纳节点的最小开销，回溯并构造路径，将 3 个端点连接在一起。三端迷宫布线的运行瓶颈仍然是波前传播，与传统的二端迷宫布线相同。

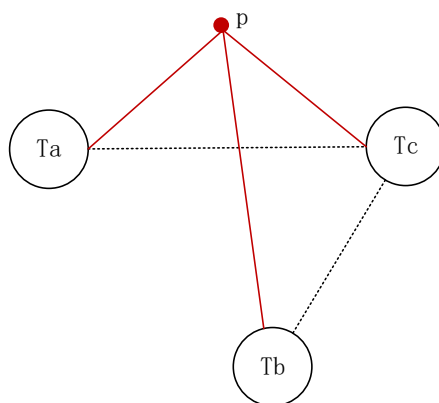


图 2-12 三端点迷宫布线

2.4 树拓扑结构生成

在全局布线初始化中，会对布线网络使用树拓扑结构生成算法，该算法主要分为两种，直线最小生成树和直线斯坦纳最小树。

2.4.1 直线最小生成树

图 2-13 展示了对于五个单元生成的直线最小生成树，它是将最小生成树中的节点距离修改为曼哈顿距离进行计算，使得生成树的边都为直线边，相比直线斯坦纳最小树，直线最小生成树因为每一条边都连接了 2 个引脚所以有更强的灵活性，所有路径都是由两个引脚连接，因此对网络进行分解时也没有中间节点。构造这类树拓扑结构应用最广泛的算法则是 Prim^[29]和 Kruskal^[30]。

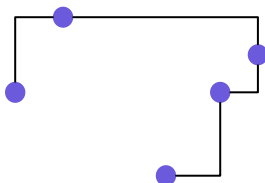


图 2-13 直线最小生成树

2.4.2 直线斯坦纳最小树

直线斯坦纳最小树算法如图 2-14 所示, 图 (a) 中展示的是直线斯坦纳最小生成树的拓扑结构, 拥有红色的斯坦纳节点和蓝色的引脚, 因此相比直线最小生成树拥有更短的线长。其中应用最广的则是非常快速且精确的直线斯坦纳最小树生成算法 FLUTE^[1]。

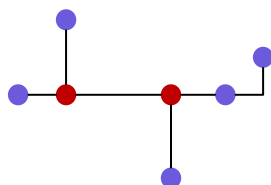


图 2-14 直线斯坦纳最小树

基于预先计算的查找表, FLUTE 算法可以解决低度的网络(最高 9 度)的最优直线斯坦纳最小树问题。对引脚数量很多的网络实现了破网技术, 该技术能递归地拆分网络, 降低网络的引脚数量, 等网络引脚数量满足要求就可以查找。根据引脚的相对位置, 所有 n 度的网络都可以划分为 $n!$ 组。对于每一组, 所有可能的布线拓扑的线长都可以写成相邻引脚之间距离的线性组合。每个线性组合被称为潜在最佳线长矢量(POWV)。FLUTE 将每个组的几个 POWV 存储到一个查找表中, 为了找到特定网络的最佳线长, 只需计算网络所属组的 POWV 对应的线长, 然后得到最小线长。FLUTE 方法只考虑沿着汉南网格的布线, 正如汉南指出的那样, 最优的 RSMT 总是可以基于汉南网格构建[84]。在生成查找表时, 它们按照 y 坐标按升序排列所有引脚的垂直序列 $s_1 s_2 s_3 \dots s_n$ 索引列表。如图 2-15 所示, 垂直顺序 1342。汉南网格中水平边的长度等于两条相邻的垂直汉南网格线之间的距离。水平边长度表示为 $h_i = x_{i+1} - x_i$, 垂直边长度表示为 $v_i = y(s_{i+1}) - y(s_i)$, 如图 2-16 所示。

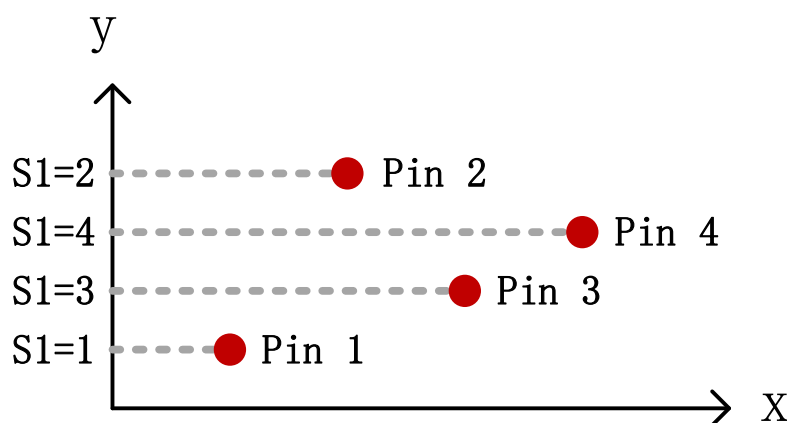


图 2-15 网络垂直序列图

对于一个给定的有四个引脚的网络，三种可能的布线解决方案的线长可以写为 $h_1 + 2h_2 + h_3 + v_1 + v_2 + 2v_3$ (图 2-17(a))和 $h_1 + 2h_2 + h_3 + v_1 + v_2 + v_3$ (图 2-17(b))。线长表示为矢量系数，称为线长矢量。图 2-17(a)和图 2-17(b)的相应线长矢量分别是(1, 2, 1, 1, 1, 2)和(1, 2, 1, 1, 1, 1)。因为它们在所有系数中都比其他向量大或相等，大多数向量是冗余的，所以只需要考虑几个有可能产生最佳线长的线长矢量。例如，在上面的三个向量中，(1, 2, 1, 1, 1, 2)可以忽略，因为向量(1, 2, 1, 1, 1, 1)生成的 v_3 更小。FLUTE 使用相同的垂直顺序将网络分组在一起，以共享一组 POWV。因为如果两个网络具有相同的垂直序列，则一个网络的每个布线解在拓扑上等价于另一个网络的解。然后，对于每个垂直方向，算法生成所有可能的布线拓扑，找到相应的线长矢量和冗余线长矢量。剩下的向量是群体的 POWV。生成所有可能的布线拓扑的一个方法是枚举汉南网格图中使用和不使用每条边的所有可能的组合，并检查生成的子图形是否包含所有引脚。但是，这种方法非常昂贵。因此，FLUTE 实现了一种基于边界压缩技术的算法。对于给定的组，他们通过压缩四个边界中的任意一个来减小网格尺寸，也就是说，将边界上的所有针移动到邻近边界的网格线上。通过将网格的布线拓扑结构扩展到原始网格，可以生成原始问题的布线拓扑集。

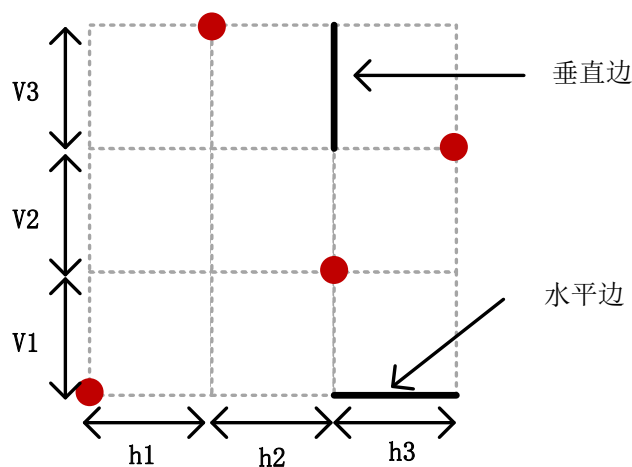


图 2-16 垂直水平边长图

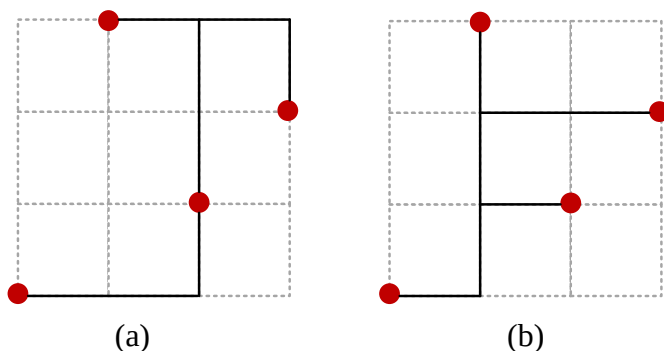


图 2-17 同个网络两种走线(a)矢量(1, 2, 1, 1, 1, 2)(b)矢量(1, 2, 1, 1, 1, 1)

2.5 协商拥塞布线

基于协商拥塞的布线^{[37]-[38]}的在寻找布线路径时会计算通过的网格边的成本，主要是使用网格边的历史拥塞数据来计算。

一开始的基于协商拥塞的布线是 McMurchie^[37]提出，以协调因为时序关键路径（critical paths）而产生的拥塞避免和最小化性能的目标。基于协商的布线现在几乎在所有的布线算法中都有使用。如公式 2-5 所示，现代布线器使用协商拥挤布线来进行拆线和重布线，其中每个边都被分配一个 $cost(e)$ 来反映边缘的需要成本。网络 N 的总成本是用于网络的所有边的成本 $cost(e)$ 值之和：

$$cost(N) = \sum_{e \in N} cost(e) \quad (2-5)$$

全更高的 $cost(e)$ 值将在更大程度上抵制边缘 e 的使用，并暗中鼓励网络寻找其他较少使用的边缘。迭代布线方法使用的方法，如 A* 搜索，以找到最少成本的路径，同时尊重边缘的容量。也就是说，在当前迭代中，所有网都是根据当前边缘的成本进行布线的。如果任何一个网络引起违规，比如某些边缘的拥塞，网就会被拆线，这些网的边缘成本会被更新以反映它们的拥塞，并在下一个迭代中重新布线。这个迭代的过程会反复地执行，直到所有需要布线的网络都被布线完成或者达到设定的结束条件。

边缘成本 $cost(e)$ 根据边缘拥塞 $\varphi(e)$ 增加，定义为使用 e 的网络的总数除以 e 的容量：

$$\varphi(e) = \frac{d(e)}{c(e)} \quad (2-6)$$

如果 e 不拥挤，即 $\varphi(e) \leq 1$ ，则 $cost(e)$ 不变。如果 e 是拥挤的，即 $\varphi(e) > 1$ ，那么 $cost(e)$ 会增加，以惩罚在后续迭代中使用 e 的网络。在协商拥塞布线中， $cost(e)$ 只能增加或保持不变。因为如果减少 $cost(e)$ ，使用边 e 的网络以前的惩罚成本将不再有效。在迭代过程中尝试布线这些网络是徒劳的，因为这些网将使用与之前布线相同的边缘。

必须控制 $\Delta cost(e)$ 的增长率范围，如果太高，整个网络会从一边推到另一边。它可以导致网络的布线在不同边之间来回跳转，导致运行时间和线长变长，并可能影响布线的成功。另一方面，如果 $\Delta cost(e)$ 太低，那么所有没有违反的网都需要太多的迭代，导致运行时间的巨大增加。理想情况下，成本增长率应该是渐进的，这样在每次迭代中，一小部分网的布线是不同的。在不同的布线器中，增长率被模拟为一个线性函数^[39]，一个动态变化的逻辑函数^[8]和一个常数缓慢增加的指数函数^[41]。在实际应用中，一个好的基于协商拥塞的全局布线器可以在保持较小的线长的同时有效地减少拥塞。

2.6 布线迭代算法

2.6.1 拆线与重布线

在确定一组网的初始布线之后，通常会发现一些布线资源被过度使用。然后，现有的布线器被撕开并重新分配到一个迭代修复框架中，这个框架称为拆线（撕裂）与重布线^{[31]-[36]}。现代的整数线性规划工具可以帮助基于整数线性规划的全球布线器在数小时内成功地完成数十万条布线。然而，商业电子设计自动化需要更大的可伸缩性和更低的运行时间。在布线阶段，如果一个网络不能被布线，通常是物理障碍或其他布线网占据了它的路径。核心思想是允许暂时的违规，直到所有的网都被发送出去，也就是说，反复地撕掉一些网，然后重新发送，以减少违规的次数。如果只有一种网格处理方法，这种方法是等价的，因此网格处理的顺序对最终解的质量有很大的影响。

对于每一个违规的网络，Kuh 和 Ohtsuki^[31]定义了拆线与重布线的成功量化概率(成功率)，并且只对最有希望的网进行改道。然而，当任何时候都不能没有违规时，成功率就需要重新计算了。这是非常昂贵的，特别是大型设计。拆线与重布线通常与其他全局布线方法结合作为后处理步骤，以进一步提高布线质量。

2.6.2 整数线性规划布线

基于整数线性规划^{[42]-[45]} (ILP) 的布线则是设置一个目标函数，并增加布线所需的约束条件，目标函数一般与总溢出和最大溢出相关，然后用 ILP 求解器进行求解。公式 2-6 是 ILP 的目标函数，2-7 则是约束条件， E 为布线网格单元边的集合，目标函数最小化目标为网格边总溢出 $\min \sum_{e \in E} \max(0, u_e - c_e)$ ， N 是布线网络集合， T_i 是网络 i 的布线候选路径集合， $x_{it} = 1$ 代表选择候选路径 t 作为网络 i 的布线路径，反之则不选择。由于布线网络数量和候选路径的增加，布线时间会呈指数及增长。因此在 ILP 布线中，关键就在于候选网络的选择，以及候选路径的生成和选择。

$$\min \sum_{e \in E} overflow_e \quad (2-6)$$

$$\begin{cases} \sum_{t \in T_i} x_{it} = 1 & \forall i \in N \\ x_{it} = \{0, 1\} & \forall i \in N, \forall t \in T_i \\ overflow_e \geq 0 & \forall e \in E \end{cases} \quad (2-7)$$

2.7 本章小结

本章首先介绍了对拥塞驱动的全局布线的问题进行了描述，随后对全局布线问题进行了建模，阐述了布线对应的技术几个衡量指标。随后描述了全局布线初始化的模式布线和网络拓扑结构生成算法，紧接着阐述了当前的迷宫布线算法以及布线迭代算法。

3 拥塞感知的全局布线算法

本章全局布线算法根据了上一章中现有的模式布线和树拓扑结构生成算法来进行初始化布线，随后针对全局布线中不可或缺的迷宫布线算法进行了优化，最后提出了新的网络级别的布线和动态布线算法，同时将两者结合形成了两阶段的布线迭代算法。通过提出的迷宫布线算法来满足布线的确定性、准确性和高速度的要求。通过提出新的网络级别并行布线来提高并行网络上限、快速对非拥塞区域的网络进行布线，然后提出动态布线调度算法在布线迭代后期来满足布线拥塞需求。

3.1 全局布线算法流程

为了解决全局布线过程中的拥塞溢出问题，本文生成了一个两阶段的撕裂重布线的全局布线框架，在不同的阶段执行不同的布线算法进行迭代，这样不仅控制住了拥塞溢出和线长，同时也对全局布线速度有了明显的提升。

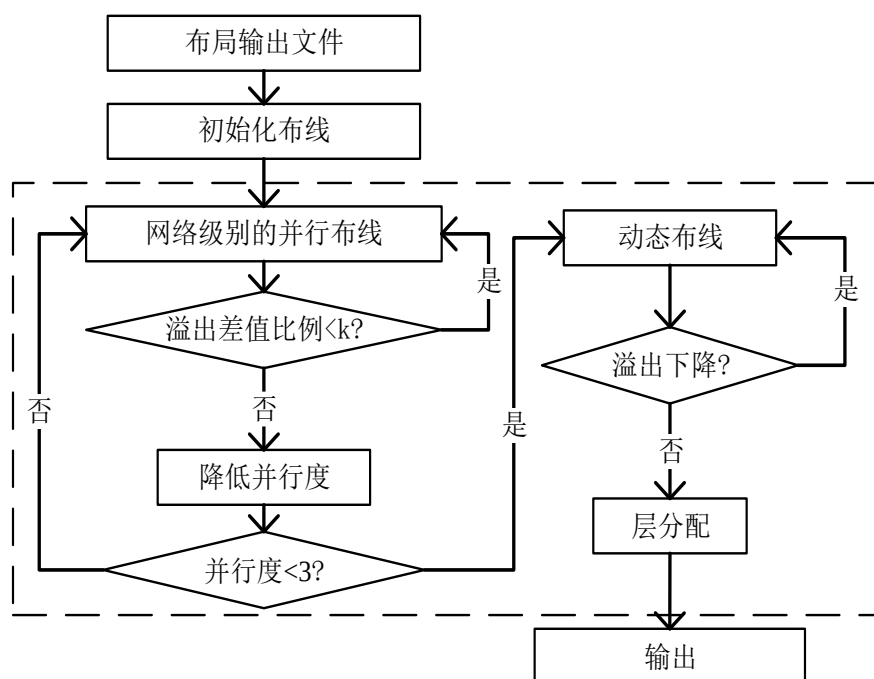


图 3-1 拥塞感知的全局布线算法流程

图 3-1 展示了拥塞感知的全局布线算法框架，芯片布局结束后的文件会作为全局布线的输入，此时将 3D 的全局布线资源映射到 2D 平面上，然后会通过直线斯坦纳树、网络分解和模式布线等操作进行初始化布线，之后再行全局布线的整体迭代。在执行布线算法时，先生成布线网络的序列，通过计算网络

的溢出数值，并根据溢出数值进行降序排序，这样使得拥塞程度更高的网络具有相对优先的布线顺序。在第一阶段执行网络级别的双路径并行布线，通过该阶段能够快速解决布线中大部分非拥塞区域的网络，当此次迭代后拥塞溢出数值 $overflow_{last} - overflow_{cur} < k * overflow_{last}$ 时，会将并行度削减，直到并行度足够小时，会切换到第二阶段的布线算法。在动态布线阶段，主要对拥塞程度较高的网络进行布线，布线结束后执行基于动态规划的层分配来把 2D 布线结果转换到 3D 布线上，最后把结果输出到布局或者详细布线。

3.2 迷宫布线算法

迷宫布线目前是所有全局布线框架中不可或缺的步骤。在迷宫布线中，全局布线利用边界盒来限制迷宫布线的搜索范围，从而达到加快布线的速度的目的。如果不能找到零溢出的布线解，则会边界盒将逐渐放松来在更大的范围内继续搜索。算法^[35]试图使用扫描操作来替代迷宫布线，但是固定的扫描次数导致越复杂的路径误差越大。而算法^[50]使用一个队列数据结构来实现迷宫布线，采用多扩展策略来保持队列数据结构处理可变的布线成本，允许一个节点被访问一次以上，但是平均只加速 18%，同时带来了不确定的质量损失。

本文为了加快迷宫布线的速度而不影响收敛性和质量提出了一种两阶段双路径迷宫布线。在 ISPD-08 adaptec2 3d 基准测试的实验统计中，平均单路径迷宫布线路径将穿越百分之四十的搜索区域并找到当前的最优路径，而本文的双路径迷宫路径将只穿越百分之二十的搜索区域。一般的双路径迷宫布线算法的终止条件是，当一方访问一个新网格节点时，该网格节点恰好在另一方的优先队列中。通过当前终止的元素的路径被认为是其解决方案。理论上，这种双路径迷宫布线算法在时间上只是迷宫布线算法的一半，即 $O(n * \log n/2)$ 。然而，这种解决方案并不一定是当前布线算法的最优解。该算法只能得到当前布线算法的次优解。该算法在易于布线的基准中可以以线长和通孔数为代价收敛，但在难布线基准中收敛性较差，布线困难。例如，在 ISPD-08 newclue7 3D 基准测试套件中，该方法的溢出比单路径迷宫布线算法的溢出高十倍以上。本文提出的两阶段双路径迷宫布线算法与普通迷宫布线算法在理论上是一致的，然而该算法在理论上比迷宫布线算法^[49]节省了 50% 的时间，在实际实验中加速效果更好。

在实验中，发现两阶段的双路径迷宫布线具有非确定性，不易于调试与工业生产，于是在其基础上又形成了确定性的双路径迷宫布线，该算法通过牺牲一部分性能来换取确定性的布线解，而不影响布线质量。

3.2.1 基于协商拥塞布线的成本函数

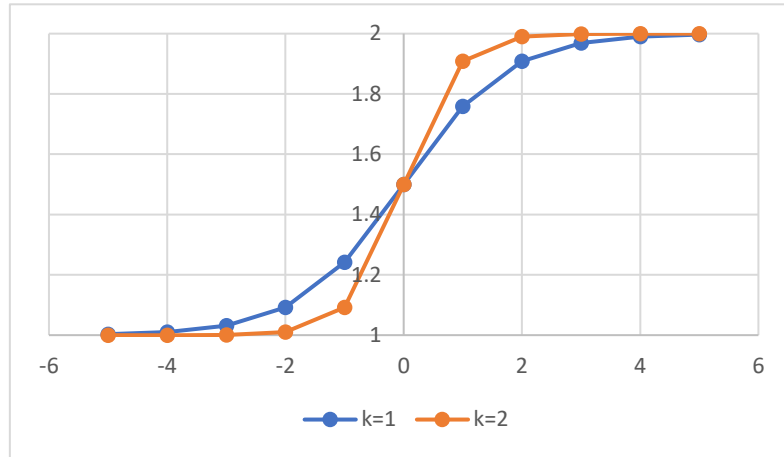


图 3-2 不同 k 值对代价函数的影响

在协商拥塞布线算法中，包括模式布线与迷宫布线中，为了寻找出非拥塞的布线路径，需要合适的成本函数指导布线算法。为了减少布线拥塞，本文采用了公式（3-2）所示的基于拥塞的成本函数， d_e 和 c_e 分别表示布线网格边的布线实际需求和布线资源容量。代价函数是 logistic 函数的变形，logistic 函数可以让布线对于需求和容量相等时附近更敏感，公式中的常量 1 代表固定的线长成本，系数 k 控制函数在 $r_e = 0$ 周围的突变速度，图 3-2 显示了 k 取值不同时的曲线，可以看出 k 值越大在中点周围的代价变化越大，在布线后期可以调大 k 值让布线对于拥塞更加敏感。 h 值初始会设置较小，以此避免过多的绕线，然后在布线过程中不断增大来提高拥塞的优先级，因为迭代时间越长代表布线拥塞越严重，这时候更大的 h 值会让线长的代价相比于拥塞更低。而 s 则是在布线后期，在拥塞边上增大对应的惩罚值。

$$r_e = c_e - d_e \quad (3-1)$$

$$cost_e = \begin{cases} \frac{h}{1 + e^{r_e * k}} + 1, & d_e < c_e \\ \frac{h}{1 + e^{r_e * k}} + 1 + \frac{h}{s * (-r_e)}, & d_e \geq c_e \end{cases} \quad (3-2)$$

3.2.2 两阶段的双路径多源多目标迷宫布线

算法分为两个阶段，在第一阶段寻找到候选路径的解，第二阶段从候选解中找到最优解。图 3-3 描述了算法的第一阶段。 PQ 网络的一端进行初始化的根据元素 $cost$ 数值维护的优先队列， $Parent$ 为了回溯时找出路径而记录每个节点的父节点， Pop_{self} 存储当前路径该节点是否被弹出优先队列的状态， Pop_{other} 则存储另一端路径下节点状态。第 2 行不断从 PQ 中取出 $cost$ 最小的元素，通过第 3 行标记该节点已经被当前路径访问，第 4-6 行表明节点如果也被另一端路

径访问则终止。第 7-17 行更新弹出元素在布线图中周围节点的 $cost$ ，如果节点已经存在于 PQ 则更新成本，不存在则插入 PQ 。

Algorithm Two-stage dual path maze routing Phase 1

```

1: while  $PQ$  is not empty do
2:    $u \leftarrow ExtractMin(PQ)$ 
3:    $Pop_{self}(u) \leftarrow true$ 
4:   if  $Pop_{other}(u) == true$  then
5:     return
6:   end if
7:   for each neighbor grid point  $v$  of  $u$  do
8:     if  $d(v) > d(u) + cost(u, v)$  then
9:        $Parent(v) \leftarrow u$ 
10:       $d(v) \leftarrow d(u) + cost(u, v)$ 
11:    end if
12:    if  $v$  is in  $PQ$  then
13:      update  $PQ$ 
14:    else
15:      insert  $v$  into  $PQ$ 
16:    end if
17:  end while
18: end while

```

图 3-3 双路径迷宫布线一阶段过程

如图 3-4 所示，图中展示了一阶段算法结束时的节点状态。节点基础状态可以分成三类：弹出元素(A_{pop}, B_{pop})、在优先队列中(A_{pq}, B_{pq})和没有被访问的节点(U)，当双路径相遇产生 $A_{pop} \cap B_{pop}$ 状态节点则返回。

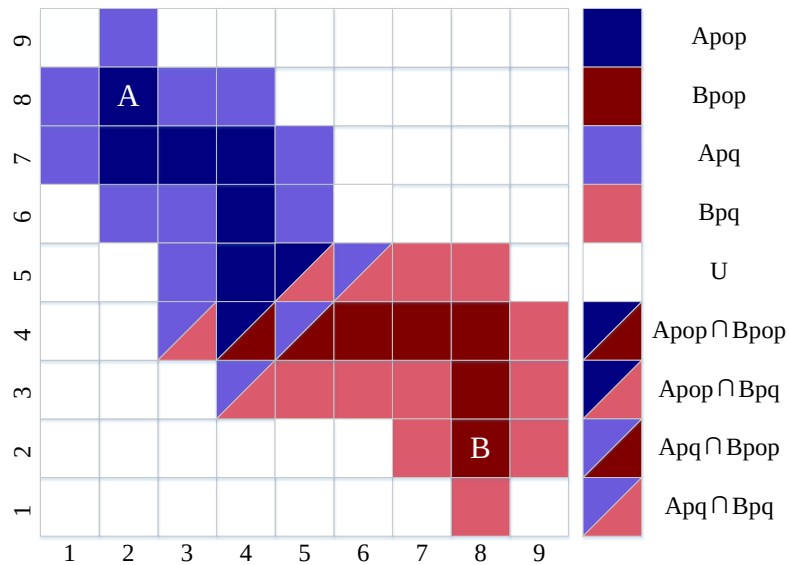


图 3-4 双路径迷宫布线节点状态

算法第一阶段结束后，此时最佳路径必然经过 $A_{pop} \cap B_{pop}$ 或者 $A_{pop} \cap B_{pq}$ 状态节点，证明如下：

一阶段算法结束后，由于起始路径和结束路径是连接的，因此路径至少通过 B 优先级队列中的一个节点，B 优先级队列节点只有三种状态： B_{pq} 、 $A_{pq} \cap B_{pq}$ 、 $A_{pop} \cap B_{pq}$ 。

设布线图中 $A_{pop} \cap B_{pop}$ 节点 v 在 A 路径成本为 $d_A(v)$ ，B 路径成本为 $d_B(v)$ ，则 $d_A(v) + d_B(v) = k$ ， k 为常数。设任意节点 u ，该节点在 B 路径上的成本为 $d_B(u)$ ，A 路径上的成本为 $d_A(u)$ ，数值不确定。

- (1) 假设最优路径或通过 B_{pq} 状态节点 u ，则 $d_B(u) = \text{常数}$ ， $d_A(u) = \infty$ 。因为 $d_A(u) + d_B(u) = \infty$ ， $d_A(v) + d_B(v) = k$ ，所以 $d_A(u) + d_B(u) > d_A(v) + d_B(v)$ 。所以最优路径不会通过 B_{pq} 状态节点，假设不成立。
- (2) 假设最优路径或通过 $A_{pq} \cap B_{pq}$ 状态节点 u 。因为 $d_A(u) > d_A(v)$ ， $d_B(u) > d_B(v)$ ，所以 $d_A(u) + d_B(u) > d_A(v) + d_B(v)$ 。所以最优路径不会通过 B_{pq} 状态节点，假设不成立。
- (3) 假设最优路径通过 $A_{pop} \cap B_{pq}$ 状态节点 u 。因为 $d_A(u) < d_A(v)$ ， $d_B(u) > d_B(v)$ ，所以 $d_A(u) + d_B(u)$ 与 $d_A(v) + d_B(v)$ 大小未知，所以假设可能成立。

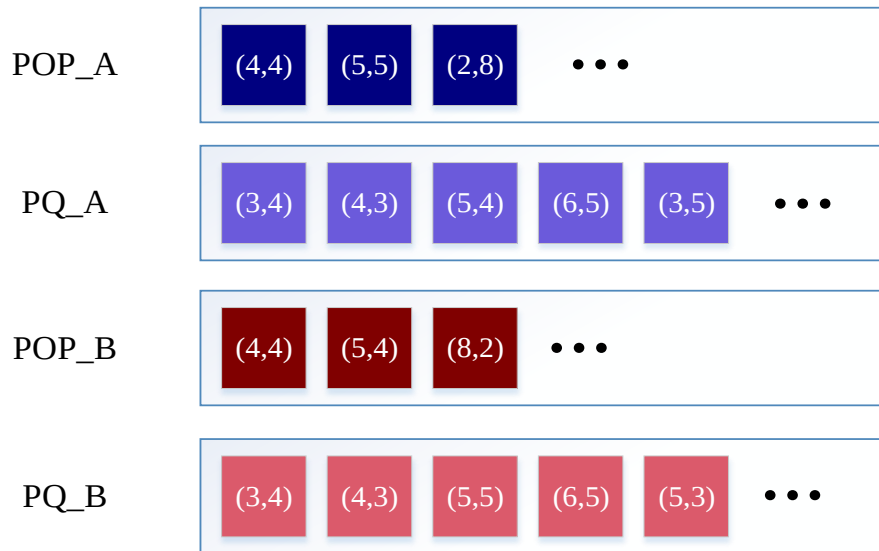


图 3-5 迷宫布线二阶段输入数据

因此最佳路径必然经过 $A_{pop} \cap B_{pop}$ 或者 $A_{pop} \cap B_{pq}$ 状态节点，此时执行第二阶段算法寻找最终解。图 3-5 展示了图 3-4 中一阶段算法结束时的队列状态， POP_A 和 POP_B 队列分别按访问顺序保存了当前 A、B 搜索过的节点， PQ_A 和 PQ_B 则是保存待访问节点的优先队列。此时，对 POP_A 和 PQ_B 的元素求交集 $POP_A \cap PQ_B = \{(4,4)\}$ ，然后使用迷宫布线访问集合 $POP_A \cap PQ_B$ 元素，求解

出 $A_{pop} \cap B_{pop}$ 和 $A_{pop} \cap B_{pq}$ 节点的成本，最终遍历元素寻找出成本最小的节点，再使用回溯找到新的布线路径。

3.2.3 确定性的双路径迷宫布线

最简洁的达到确定性解的方案就是，双路径上轮流搜索固定的步数，但是这种耗费时间的同步在实际运行中是无法容忍的。本文在双路径迷宫布线的一阶段后面额外增加回退、继续布线两步，让每次布线两边都走同样的步数，以此来达到确定性解的效果。

图 3-6 (a)展示了理想的双路径迷宫布线的一阶段终止时的搜索状态，图 3-6(b)展示了实际布线终止时的已搜索范围，圆形区域代表两个路径上已经搜索的布线范围。通过回退步骤让蓝色方向把搜索范围变为图 3-6 (c)中的虚线部分，与红色实线范围一致，此时仍然会保留蓝色虚线与实线之间的数据，然后两边轮流走 1 步，由于蓝色区域只需要复用数据，所以等同于只在走红色路径。最后红色与蓝色虚线相交，如图 3-6 (d)所示，形成了和图 3-6 (a)一样的理想状态。

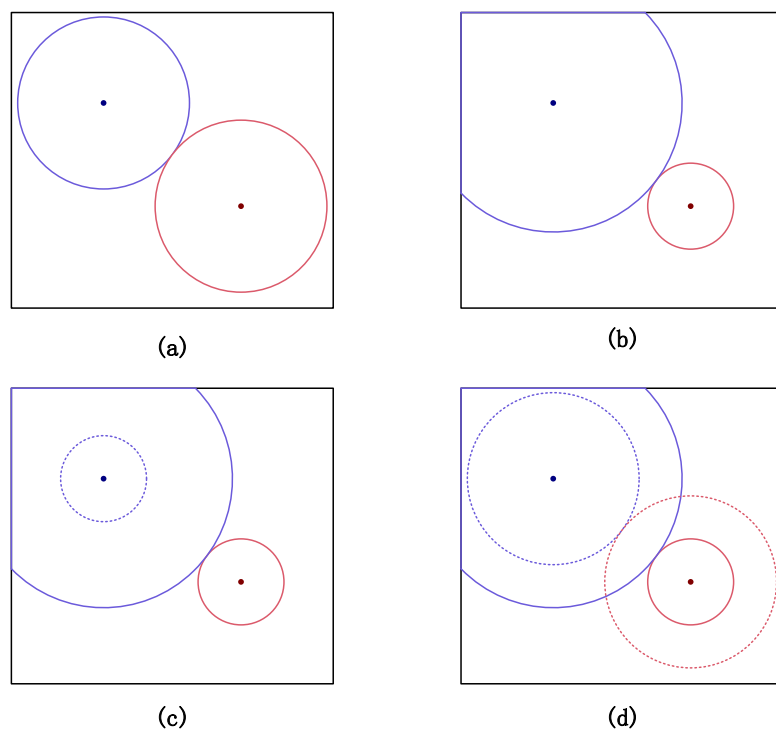


图 3-6 不同迷宫布线算法搜索状态

下面证明解是确定性解。

已知：A、B 两个优先队列，初次相遇弹出元素数量分别为 P_A 、 P_B ，状态分别为 A_{P_A} 、 B_{P_B} 。

假设 A、B 依次弹出元素，状态为 A_1 、 B_1 ， A_2 、 B_2 ， A_3 、 B_3 ……终止位置必然固定，假设为 A_{mid} 、 B_{mid} 。

设 $P_A > P_B$ ，则将 A 回溯到弹出元素数量为 P_B 的状态 A_{P_B} ，

若此时满足终止条件:则 A、B 状态为 A_{mid} 、 B_{mid} 。

若此时不满足终止条件: A、B 依次弹出元素状态更新为 A_{P_B+1} 、 B_{P_B+1} ， A_{P_B+2} 、 B_{P_B+2} ， A_{P_B+3} 、 B_{P_B+3} ……必然会在 A_{mid} 、 B_{mid} 相遇。

确定性的双路径迷宫布线算法由图 3-3 算法、图 3-7 算法和两阶段迷宫布线第二阶段算法依次构成。在图 3-7 中， $Index_{other}(u)$ 记录节点在另一端路径中被探索到时的步数。第 1 行计算双路径中较小的步数，6-7 行表明节点 u 在另一端路径 $step$ 步数内被搜索到，其他与之前的算法几乎一致。

Algorithm Deterministic maze routing backoff algorithm

```

1:   $step = \min(step_A, step_B)$ 
2:  while  $step \leq \max(step_A, step_B)$  do
3:       $step = step + 1$ 
4:       $u \leftarrow ExtractMin(PQ)$ 
5:       $Pop_{self}(u) \leftarrow true$ 
6:      if  $Pop_{other}(u) == true$  and  $Index_{other}(u) \leq step$  then
7:          return
8:      end if
9:      for each neighbor grid point  $v$  of  $u$  do
10:         if  $d(v) > d(u) + cost(u, v)$  then
11:              $Parent(v) \leftarrow u$ 
12:              $d(v) \leftarrow d(u) + cost(u, v)$ 
13:         end if
14:         if  $v$  is in  $PQ$  then
15:             update  $PQ$ 
16:         end if
17:     end while
18: end while

```

图 3-7 确定性迷宫布线回退算法过程

3.3 布线迭代算法

3.3.1 网络级别的并行布线

在强约束的布线中，需要在统一的布线资源图中依次对网络进行布线，这样具有较好的收敛速度，但是速度较慢。或者采用最大独立集的方式计算出相交的网络来增加同时布线的网络数量，但是实际布线的范围最坏的情况下能扩展到整个布线区域，所以如何确定不相交的网络是一个难以解决的问题。

由此产生了弱约束的布线，如图 3-8 所示，让 N 个网络共享一个布线资源图，并同时布线。He J 等人^[40]就使用了这种布线方式，但是缺点也很明显，同时布线的网络数量越多布线就越难以收敛，当 N 等于网络数量时，在正常的布线基准中布线将无法收敛。本文在前期也采用了弱约束的并行布线，但是为了提高收敛度，在其中融入了前一小节的提出的双路径布线，由此在相同的线程数量下，能够减少一半的并行网络数，极大地提高了收敛性。在此基础上，本文设计了对应的并行度下降算法来逐渐恢复成强一致性，算法包含上一次迭代后的总拥塞溢出数值 $overflow_{last}$ 和此次布线溢出 $overflow_{cur}$ ，然后计算出两者差值比例

$$ratio = (overflow_{last} - overflow_{cur}) / overflow_{last} \quad (3-3)$$

当 $ratio < k$ 时， $N = N/2$ ，直到 N 小于 3 时，此阶段布线结束。

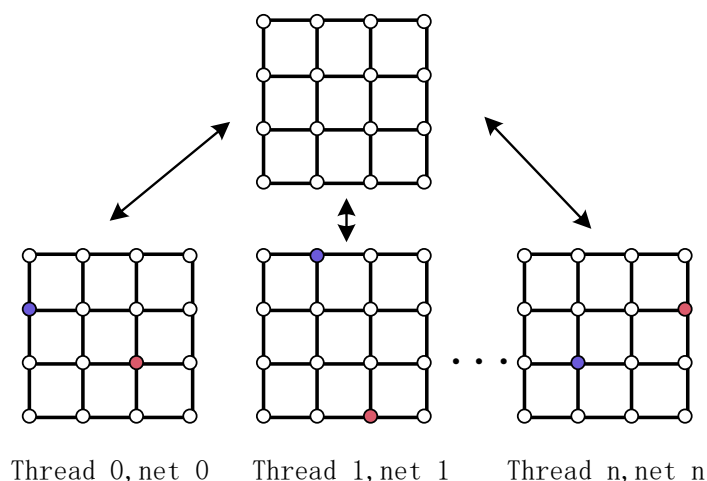


图 3-8 网络级别布线

3.3.2 动态布线调度算法

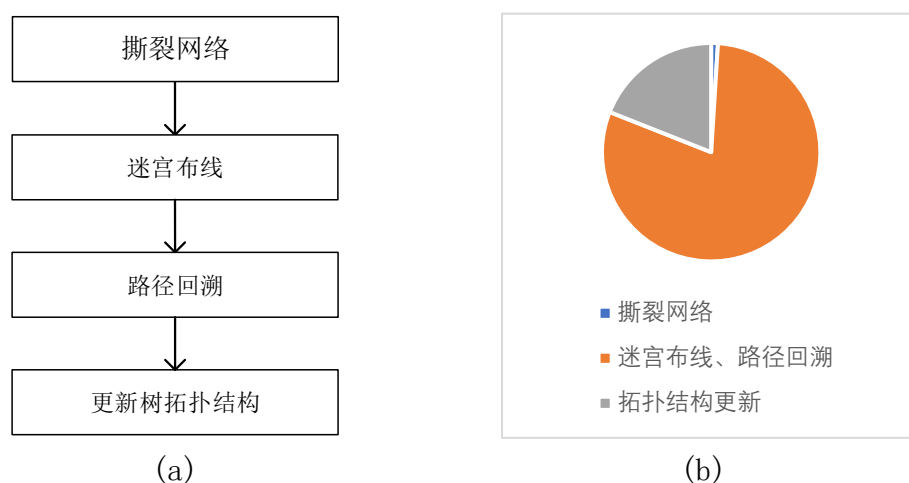


图 3-9 (a) 单次迭代流程 (b) 各个部分 adaptec1 中时间占比

在 FastRoute 4.1^[49]的算法中，每次全局布线迭代开始时，穿过不同全局布线单元的网络都已经存在上一次布线后的路径。此时对于需要布线的网络执行图 3-9 所示的步骤，对于需要重布线的网络进行撕裂，然后初始化迷宫布线的数据，紧接着进行迷宫布线，之后通过布线的中间数据路径回溯来获取新的布线路径，最后更新网络斯坦纳树的拓扑结构。在 ISPD-08 adaptec1 3D 基准的实验中，发现撕裂网络花费的时间几乎为零，而迷宫布线和路径回溯的时间占用了 80% 的运行时间。先前关于布线加速的研究尝试主要集中于并行布线启发式的开发。Liu W.H^[18]等人提出了一种混合单边单调布线算法，NCTU-GR 2.0^[14]提出了一种有界长度迷宫布线（BLMR）算法，并集成到多线程冲突感知全局布线器中。然而，这些方法会显著降低解决方案的质量，明显影响布线的拥塞。SPRoute^[40]提出细粒度的并行，然而由于细粒度并行算法自身的缺陷，电路的拥塞越集中算法效果越差。在图 3-10 中，图 (a) 表示在当前布线拥塞状态下单个双引脚网络的布线路径，而在图 (b) 中展示了布线资源改变导致布线拥塞变化后该网络新生成的布线路径。基于该思想，本文提出了动态布线调度算法，既满足了布线中的拥塞问题，同时也无损地提升了布线速度。

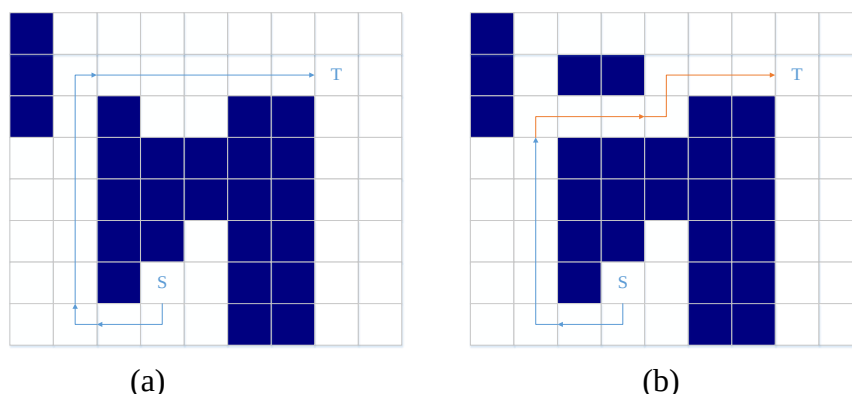


图 3-10 布线拥塞改变前后的双引脚网络的布线路径图

(1) 布线初始化

在动态调度算法中，同时布线的每个网络都拥有独立的布线资源，另外存在唯一的全局布线资源，如图 3-11 所示。

为了节省计算机内存资源，该算法根据当前迭代算法设置的线程数量创建相应数量的一组布线资源。然后，全局布线资源的数据被拷贝到每个独立布线资源中进行初始化。接下来，初始化每个布线单元的状态，并将该组布线资源以双向循环链表的形式连接起来，使其首尾相接。

随后，算法遍历所有需要进行布线的网络。通过检查各个网络的边是否穿过全局布线网格的拥塞边，并计算出每个网络在全局布线资源上的拥塞情况，以确定各个网络的总体拥塞程度。然后，根据降序排序，生成一个固定顺序的布线网络序列。

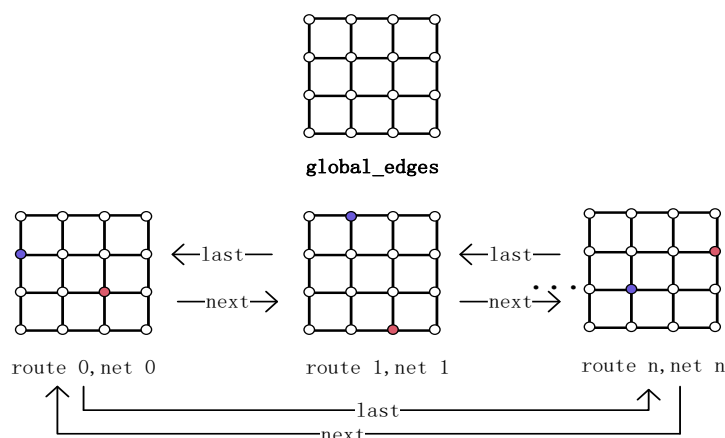


图 3-11 动态布线资源图

(2) 调度算法设计

在撕裂与重布线的算法框架中，每一个网络的撕裂只消耗固定的时间 $O(k)$ ， k 代表网络对应路径的长度。而在重布线阶段中的迷宫布线算法的时间复杂度为 $O(n \log n)$ ， n 代表布线包围盒中网格的数量。因此，为了减少网络间的冲突数据，顺序执行 *rip_up* 操作，对于重布线则采用并行的方式。如图 3-12 所示，线程队列由三个线程 A、B 和 C 组成，它们分别布线红色网络、蓝色网络和绿色网络。为了确保撕裂网络可以序列化执行，线程 A 将撕裂后面所有线程的红色网络，而线程 B 将在线程 A 撕裂红色网络之后，撕裂后面的所有蓝色网络，线程 C 与之相同。这样如果网络间产生冲突，次要网络则不会受到前面网络原有路径的影响，能有效减小次要网络受影响的范围。

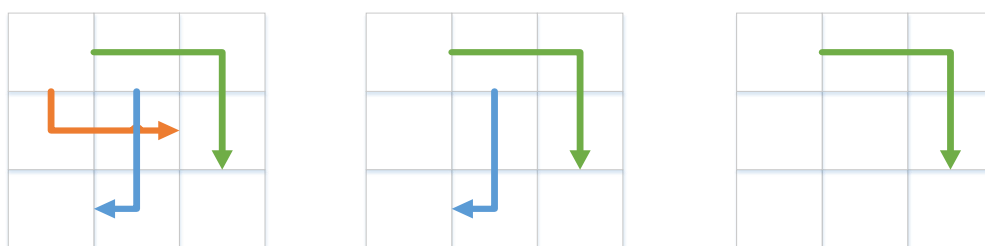


图 3-12 布线拆线序列化

调度方法的工作原理使用了一个双线程的作为案例来说明，可以轻松地扩展到多线程或多线程工作区。假设有两个线程 t_A 和 t_B ，各个线程分别占有网络 net_A 和 net_B 。 t_A 当前是主线程，而 t_B 是次线程。这两个网络都已被撕裂并释放了它们所占用的布线资源。然后，它们同时用新路径 p_A 和 p_B 重新布线， t_A 将首先使用提交 p_A 更新阻塞图，随后 t_B 将令牌传递给 t_B ，在那里他们交换了主要的次要身份。如果与 p_A 没有路径冲突，则将立即提交 p_B ，否则， p_B 将根据冲突数据部分进行撕裂并重新布线。

Algorithm Dynamic scheduling algorithm

```

1:  for each net  $n$  in  $nets$  do
2:      for each edge  $e$  in  $n$  do
3:           $waitLastRipup()$ 
4:          if  $needRipUp(e)$  and  $RipUpGlobal(e, edge\_changes)$  then
5:               $routes.sub(edge\_changes)$ 
6:               $isripup \leftarrow true$ 
7:               $mazed()$ 
8:               $global\_edges.add(new\_path)$ 
9:              for each  $route$  in  $routes$  do
10:                   $route.edges.add(new\_path)$ 
11:                  if  $route \neq this$  then
12:                       $route.notify(new\_path)$ 
13:                  end if
14:              end for
15:               $update\ edges\ by\ routes.edge\_changes$ 
16:               $route_{primary} \leftarrow this.next$ 
17:               $updatetree(n, e)$ 
18:          end if
19:      end for
20: end for

```

图 3-13 动态调度算法设计

调度算法如图 3-13 所示，多个布线单元 $route$ 通过使用同步算法取出有效网络，随后等待前一个布线单元撕裂网络，主布线单元例外。然后通过计算网络边的路径的拥塞数值确定是否需要撕裂网络边，需要撕裂则会移除网络边占用的布线资源和全局布线资源并将修改数据存放到 $edge_changes$ 中，接着沿链表顺序遍历当前布线单元的后续单元，并将当前布线单元修改的边资源数据依次修改到与当前资源数据一致。通过设置 $isripup$ 为真来保证布线单元之间的撕裂操作的序列化，随后在初始化动态布线数据后执行动态布线，在动态布线中会复用搜索过的路径并解决每个布线单元与全局布线资源之间冲突，最后通过每个节点保存的 $Parent$ 数据使用回溯算法获取到当前布线网络的新布线路径。然后使用 new_path 更新全局布线资源，同时传递 new_path 到其它的布线单元，并通知其它布线单元解决布线资源冲突更新布线数据，其它布线单元则会在此时对于搜索状态进行更新。接下来的算法只有主布线单元才能执行，此时需要遍历其它布线单元的 $edge_changes$ ，并将 $change$ 更新到自身的布线资源中，设置下一个布线单元为主布线单元，以此放置到布线单元列表末尾，最后对网络的斯坦纳树结构进行更新。

此外，本文将动态布线与双路径迷宫布线进行了结合，缓解了布线冲突。如图 3-14 中，图 (a) 和图 (b) 代表普通布线能够利用的路径，而图 (c) 和图 (d) 中的双路径迷宫布线可以有效地减轻重新布线的工作量。这种双路径的布线算法从各自的网格集合开始寻找，当一条布线路径受到严重影响，则另一条路径大概率受影响较小。

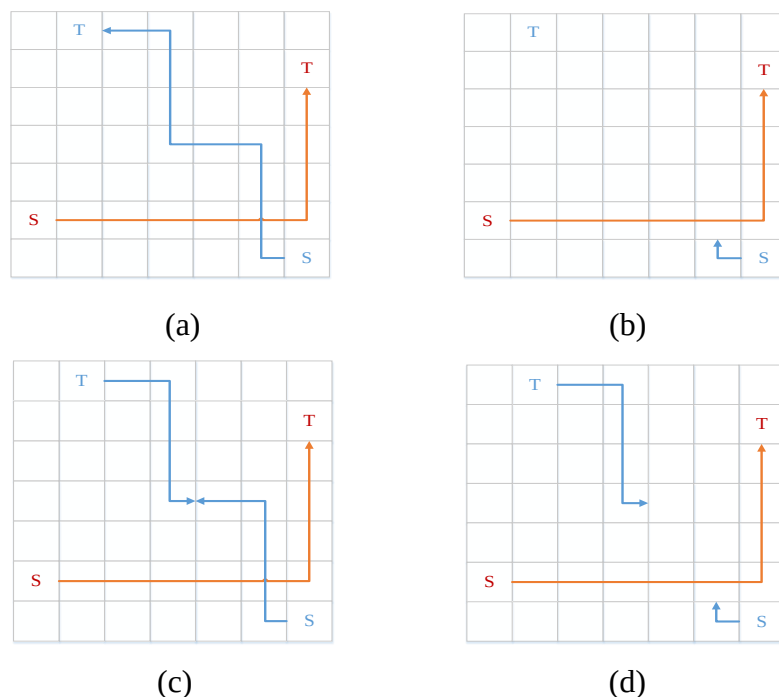


图 3-14 动态调度与双路径布线结合

(3) 网络冲突解决

如图 3-15 所示，图 (a) 展示了当前布线网络的状态，深蓝色为已经探索的范围，浅蓝色为处于优先队列中的待探索范围，图 (b) 则是上一个布线单元网络的新布线路径。图 (a) 的布线网格状态在受到图 (b) 布线路径的影响后，布线网格状态换成了图 (c) 的网格状态。图 (a) 中的数字代表该节点的探索序列号，节点 17 受到布线路径影响后，导致后续序列号的节点被重置。

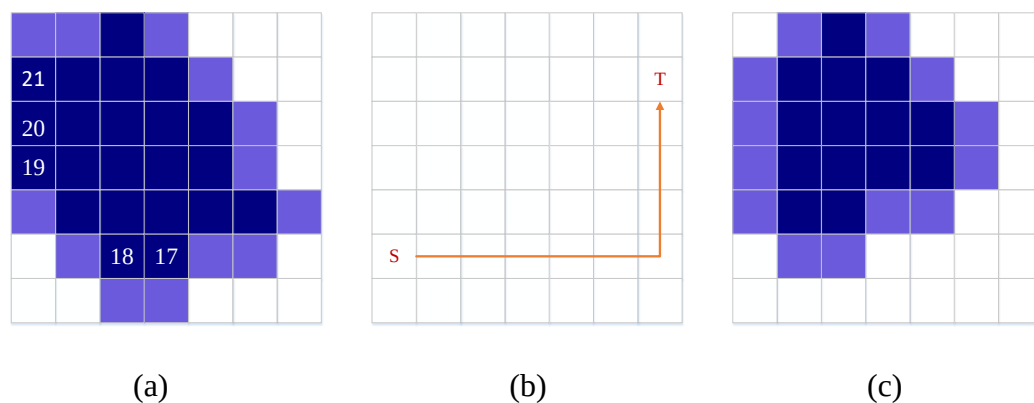


图 3-15 布线网络状态变化图

图 3-16 展示了布线网络冲突时的解决算法伪代码，在动态布线调度算法中检测到自身的网格边资源产生变化，同时自身不是主布线单元时，则会执行该算法解决布线资源冲突。*new_path*是冲突的路径，*Index(u)*记录了*u*搜索顺序的索引，*d(u)*表示到节点*u*的总成本。首先找到冲突最小索引节点*min_index*，如果*min_index*是无效索引，则代表当前搜索范围没有受到冲突路径影响，随后对*min_index*索引之后的所有节点状态和对应索引进行重置，最后对无冲突集合中的边界节点进行遍历，生成新的优先队列集合和节点成本。

Algorithm Resolving routing conflict nets

```

1:  for each grid u in new_path do
2:      if Index(u) < min_index and d(u) ≠ 0 then
3:          min_index = Index(u)
4:      end if
5:  end for
6:  for i in range(min_index, pop_grids.size) do
7:      u ← pop_grids(i)
8:      u.reset()
9:      Index(u) ← invalid
10: end for
11: pop_grids.resize(min_index)
12: PQ.clear()
13: for each grid u in pop_grids do
14:     for each neighbor grid point v of u do
15:         if Index(v) is invalid and d(v) > d(u) + cost(u, v) then
16:             update PQ and d(v)
17:         end if
18:     end for
19: end for
  
```

图 3-16 解决布线冲突网络

3.4 本章小结

本章介绍了本文提出拥塞感知的全局布线算法总流程，然后详细描述了布线算法流程中提出的迷宫布线算法并进行了算法有效性证明，随后叙述了本文改进的网络级别的并行布线，最后阐述了动态布线调度算法的实现细节。

4 实验结果与分析

本文提出的算法采用 C++ 语言进行实现，运行在 4.10GHz 英特尔酷睿 i5-10600KF 处理器和 32GB 内存的机器上。本章先介绍了国际物理设计研讨会举办的竞赛的 ISPD-08 基准测试案例数据，随后对本文算法的有效性和拥塞解决能力进行了验证，最后与竞赛上当前学术中表现最优秀的开源全局布线算法对比了拥塞溢出解决能力和布线效率。

4.1 基准测试案例

表 4-1 ISPD-08 基准测试案例统计数据表

Name	Grids	Layers	Nets	Routed Nets	Max Deg	Avg Deg
adaptec1	324 X 324	6	219K	177K	340	4.2
adaptec2	424 X 424	6	260K	208K	153	3.9
adaptec3	774 X 779	6	466K	368K	82	4
adaptec4	774 X 779	6	515K	401K	171	3.7
adaptec5	465 X 468	6	867K	548K	121	4.1
bigblue1	227 X 227	6	283K	197K	74	4.1
bigblue2	468 X 471	6	557K	429K	260	3.5
bigblue3	555 X 557	8	1.12M	666K	91	3.4
bigblue4	403 X 405	8	2.23M	1.13M	129	3.7
newblue1	399 X 399	6	332K	271K	74	3.5
newblue2	557 X 463	6	463K	374K	116	3.6
newblue3	973 X 1256	6	552K	442K	141	3.2
newblue4	455 X 458	6	636K	531K	152	3.6
newblue5	637 X 640	6	1.26M	892K	258	4.1
newblue6	463 X 464	6	1.29M	835K	123	3.8
newblue7	488 X 490	8	2.64M	1.65M	113	3.6

基准测试案例是评价算法好坏的测量标准，不同领域都有各自的基准测试案例。本文使用的基准测试案例是 ISPD-08 比赛提供的一组套件，专门用来测试拥塞避免的全局布线算法性能。国际物理设计研讨会（ISPD）提供了一个主要论坛，以交流思想并促进物理设计各个方面的创新研究，从 ASIC 和 FPGA 设计的传统主题到影响集成电路（IC）物理设计的新兴技术。ISPD 每年都会根据工业界的一些难题来举行竞赛，该竞赛是芯片后端物理设计的顶级比赛。

ISPD-08 布线竞赛包含了来自工业 ASIC 设计的 16 个基准，其中 8 个测试案例来自 ISPD-07。在 ISPD-07 的竞赛中，单个通孔成本是单位线长的三倍，而在 ISPD-08 竞赛中，通孔与线长单位成本相同，本文也采用了 ISPD-08 的比赛规则。基准测试案例如表 4-1 所示，对于每个测试案例，包含了布线区域划分的网格数量（Grids）、可以布线的层数（Layers）、总网络数量（Nets）、实际需要布线网络数量（Routed Nets）、单个网络最大引脚数量（Max Deg）以及平均每个网络的引脚数量（Avg Deg）。

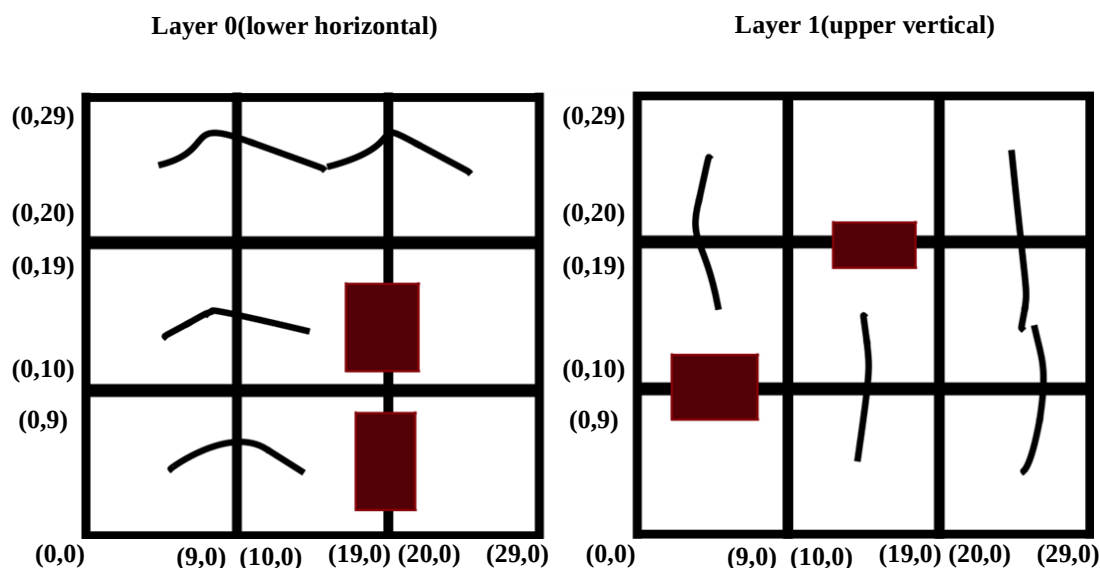


图 4-1 两层布线层的三维布线示意图

如图 4-1 所示，这是一个具有两个布线层的三维布线示意图。每层的图中的网格边容量为 1，第一层只有水平容量，而第二层只拥有垂直容量。在图中的深色矩形块就是阻塞区域，该区域不能布线。所有基准都有多个金属层（三维），并带有额外的拥塞信息。输入文件格式是迷宫格式的变体。文件将是普通的 ASCII 文本，指定全局路由图的大小和形状、图边缘的容量、信号网络的数量、引脚数量及其在每个网络中的位置。文件格式的设计使解析变得容易。输出文件格式将是 BoxRouter^[10]格式的变体。选择输入/输出格式是为了让许多学术工具能够参加比赛，而编写使用这种格式的全局布线器并不会特别困难。

图 4-2 展示了全局布线竞赛的输入文件格式，注释为斜体（这些注释不会出现在实际输入文件中）。此示例说明了五个布线层的问题。与原始迷宫格式相比，有许多变化：

当原始迷宫格式假定为单个布线平面时，指定了布线层数。进行此更改是为了允许在全局布线期间执行层分配。每个层可以在每个全局布线中的每个方向上具有唯一的容量，并且对于水平和垂直方向可能不同。优先的布线方向将通过在非优先方向上具有零容量来给出。在垂直和水平容量线中，数字表示了

数字对应序号布线层的布线容量，还指定了会影响容量计算的最小导线宽度和最小导线间距。

指定全局布线区域的左下角（最小 X 和 Y 坐标），以及每个布线网格的宽度（X 维度）和高度（Y 维度）。此外，坐标位置以 XY 位置而不是布线区域相对位置给出。这一变化的动机是为了将来能够基于这些基准进行详细的布线竞赛。可以使用 $\text{floor}(\text{pin_x} - \text{lower_left_x})/\text{tile_width}$ 和 $\text{floor}(\text{pin_y} - \text{lower_left_y})/\text{tile_height}$ 来完成从引脚位置到网格编号的转换。引脚不会在全局布线单元的边界上而是在规定的布线区域范围之内。

每个网的最小布线宽度将跨越所有层。布线时，通过将所有网络交叉和边缘的宽度加上最小间距乘以网络数量，计算全局布线图边缘的利用率。这比迷宫模型更复杂，但更准确地反映了现代布线挑战。每条导线都需要满足相邻导线之间的间距约束，将其视为在金属线的每一侧保留一半的最小间距。

通过包括容量调整对阻塞进行建模。在全球布线基准中，可能存在障碍或预先布线的金属线。为了将其传递到全局布线器，成对的（相邻的）全局布线网格可能具有不同于基准文件开始时指定的默认值的容量。

容量的计算比典型的学术全球布线工具更复杂。每个全局布线网格边将具有一个容量，这是可用空间的度量，而不是全局布线轨道的数量。如果最小导线宽度为 20，最小间距为 10，并且网格边缘的容量为 450，则这对应于 15 个最小宽度路径（ $15 * (20 + 10)$ ）。一般来说，希望网格边的布线利用率低于容量，因为更高的利用率可能更难让详细布线器完成。

<i>grid # # # (x grids, y grids, number of layers)</i>
<i>vertical capacity # # # # (vertical capacity by default on each layer)</i>
<i>horizontal capacity # # # #</i>
<i>minimum width # # # #</i>
<i>minimum spacing # # # #</i>
<i>via spacing # # # #</i>
<i>lower_left_x lower_left_y tile_width tile_height</i>
<i>num net #</i>
<i>netname id # number_of_pins minimum_width</i>
<i>x y layer</i>
<i>x y layer</i>
...
<i>[repeat for the appropriate number of nets]</i>
<i># capacity adjustments (to model congestion)</i>
<i>column row layer column row layer reduced_capacity_level</i>
<i>[repeat for the number of capacity adjustments]</i>

图 4-2 全局布线输入数据格式图

4.2 算法验证

在实验中，本文首先验证了对^[40]改进后的网络级别布线的有效性，随后展示了本文布线算法的拥塞溢出解决能力和可伸缩性。

(1) 网络级别布线有效性验证

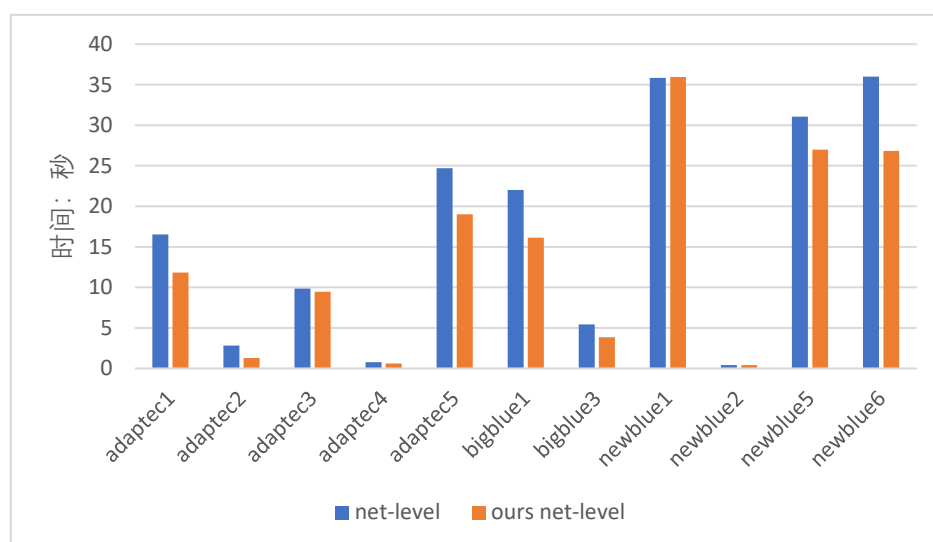


图 4-3 不同网络级别布线速度对比图

本文使用了 **net-level**^[40] 和自己提出的网络级别的布线，嵌入到之本文的全局布线框架中，对于全局布线第一阶段进行替换，其他保持不变。图 4-3 中展示了 ISPD -08 中全局布线后拥塞溢出为零的基准测试案例，纵坐标则是两个布线阶段所消耗的时间（单位：秒），都使用了 12 线程，可以看出本文的算法几乎完全优于 **net-level**^[40]。由于算法^[40]天然的缺陷，随着并行网络数量的增加，会呈现出假并发的趋势，导致算法不能有效利用多核的性能，而本文的算法虽然没能完全解决这个问题，但极大地缓解了这种趋势，相比算法^[40]几乎能利用到翻倍的多核性能。

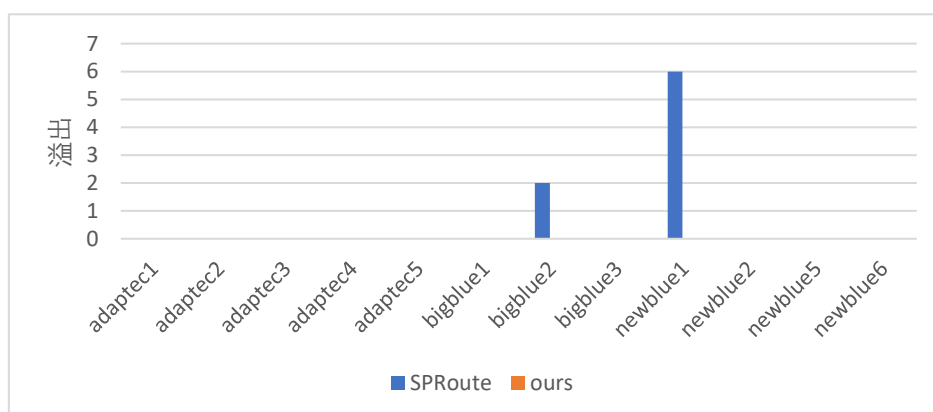


图 4-4 易布线基准案例溢出对比图

(2) 拥塞溢出验证

在此处，本文将 ISPD-08 基准测试分为易布线基准和难布线基准两种，在易布线就基准测试上，本文的算法能够将拥塞溢出数值能够完全收敛到 0，也就是完全解决了拥塞问题，而未能够完全解决拥塞的案例则分到了难布线基准测试中。在图 4-4 中，横坐标是易布线基准测试案例，纵坐标是拥塞溢出数值，可以看到本文的算法在所有案例中都解决了拥塞问题，而 SPRoute^[40]则在 bigblue2 和 newblue1 中有着不稳定的质量表现，会出现拥塞无法解决的情况。而难布线基准测试案例如图 4-5 所示，本文与 SPRoute 之间的拥塞溢出相近。



图 4-5 难布线基准案例溢出对比图

(3) 可伸缩性

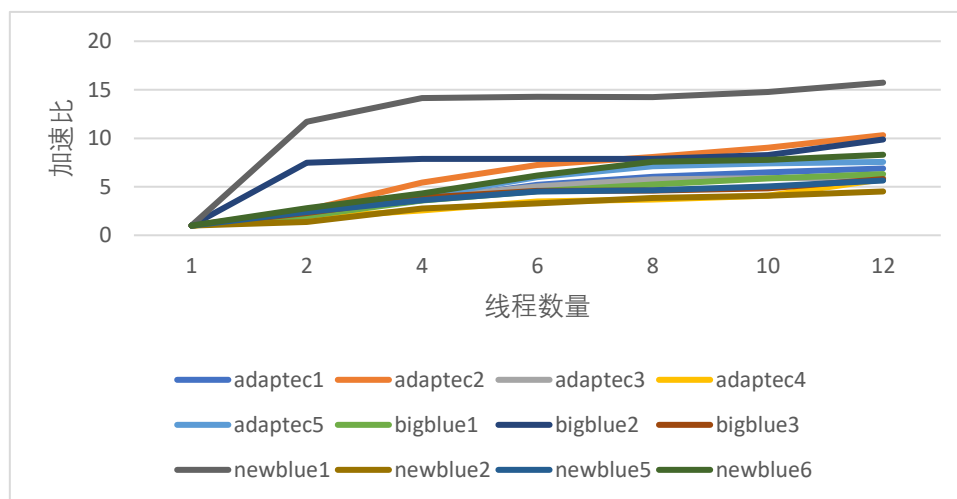


图 4-6 无溢出基准案例的可伸缩性图

由于本文的算法是多线程算法，所以对于算法多核性能利用的可伸缩性进行了测试，如图 4-6 所示，这里都是在本文的算法中拥塞溢出完全收敛到 0 的基准测试案例，图中绘制了随着线程数量的增加，各个测试案例以一个线程的迭代执行时间为基准的加速比。从中可以看出，当使用 12 个线程时，测试基准

的平均可伸缩性为 7.11，而其中最差的加速效果也有 4.5 倍，而最好的效果则达到了 15 倍之多。

4.3 与优秀算法对比

在实验中，本文对比了两个学术上在 ISPD-08 基准测试案例中表现最优秀的开源布线算法 SPRoute^[40]和 FastRoute 4.1^[49]以及一个公开二进制文件的学术布线器 NCTU-gr 2.0^[14]，分别比较了布线拥塞溢出、总线长和布线迭代的运行时间，最后再分析了本文算法的优势。在实验中，对于多线程的布线算法，本文连续运行了五次布线算法，然后计算了各个指标的平均值。在评估能同时指导布局 and 详细布线的全局布线算法的解决方案质量时，应优先考虑其控制拥塞溢出的能力，其次是运行时间，而线长在拥塞估计中应作为最后考虑的因素。

(1) 拥塞溢出

表 4-2 与三种布线算法的拥塞溢出结果比较

Name	Ours	SPRoute	FastRoute 4.1	NCTU-gr 2.0 Fast	NCTU-gr 2.0 Regular
adaptec1	0	0	0	0	0
adaptec2	0	0	0	0	0
adaptec3	0	0	0	0	0
adaptec4	0	0	0	0	0
adaptec5	0	0	0	0	0
bigblue1	0	0	0	0	0
bigblue2	0	6	0	0	0
bigblue3	0	0	0	0	0
bigblue4	172	136	278	1056	512
newblue1	0	6	4	492	0
newblue2	0	0	0	0	0
newblue3	31277	31272	31338	36850	37182
newblue4	172	140	138	216	172
newblue5	0	0	0	0	0
newblue6	0	0	0	0	0
newblue7	98	136	60	792	286
sum	0	-23	99	7687	6433

根据表 4-2 的结果，在 16 个基准测试案例中，本文的算法在 12 个测试案例中稳定地产生了无拥塞溢出的解。在这 12 个测试案例中，SPRoute 在 bigblue2 和 newblue1 测试案例上出现了拥塞无法完全解决的情况，FastRoute 4.1 在 newblue1 测试案例上未能解决拥塞，NCTU-gr 2.0 在 Fast 模式下无法解决

newblue1 测试案例的拥塞。在剩余的 4 个无法完全解决拥塞溢出的测试案例中，本文的算法的拥塞溢出结果均优于 NCTU-gr 2.0 的 Fast 和 Regular 模式下的解；对于 FastRoute 4.1，本文算法在 bigblue4 和 newblue3 测试案例上的解质量更好；而在 newblue7 测试案例上的拥塞溢出解优于 SPRoute。总体而言，以本文算法结果为基准，SPRoute 算法在测试案例上的结果有胜有负，而其他三种算法的结果均不及本文算法。

(2) 总线长

表 4-3 展示了各个算法的总线长比较，单位为 10^5 。从表中可以看出，NCTU-gr 2.0 在 Fast 模式下的总线长最差，本文的算法的总线长度结果排名第二，与第一名的结果相近。由于本文目标是设计出快速且能准确估计布线拥塞的全局布线算法，因此没有在线长进行额外优化。

表 4-3 与三种布线算法的总线长结果比较

Name	Ours	SPRoute	FastRoute 4.1	NCTU-gr 2.0 Fast	NCTU-gr 2.0 Regular
adaptec1	54.879	54.045	53.998	58.208	54.441
adaptec2	52.256	52.231	52.352	54.075	52.681
adaptec3	131.558	130.933	131.578	136.265	131.116
adaptec4	121.262	121.251	121.672	123.100	121.941
adaptec5	157.193	156.100	156.291	166.195	159.467
bigblue1	58.134	57.483	56.970	64.993	59.727
bigblue2	91.498	92.406	92.633	94.971	90.990
bigblue3	130.152	130.124	130.574	133.722	131.677
bigblue4	230.635	230.345	231.443	240.482	231.704
newblue1	46.627	47.232	47.584	48.109	47.002
newblue2	75.125	75.121	75.533	76.635	75.886
newblue3	109.503	110.710	109.493	112.809	108.048
newblue4	131.277	130.843	130.636	134.427	129.910
newblue5	232.362	231.339	232.332	242.337	234.576
newblue6	179.551	178.480	178.626	193.247	182.732
newblue7	356.299	353.704	355.017	362.092	352.88
avg	1	0.998	1.000	1.042	1.04

(3) 运行时间

根据表 4-4 的结果，表中比较了几种算法的布线迭代时间，时间单位为秒。可以看出，在 16 个基准测试案例中，本文的算法的运行时间都比 FastRoute 4.1 快，并且在 15 个测试案例中比 SPRoute 和 NCTU-gr 的 Regular 模式下快，在 9 个测试案例中的运行速度优于 NCTU-gr 的 Fast 模式。总体而言，

本文的算法在所有测试案例上的平均加速效果是最好的，相比第二名的 SPRoute，加速了 1.8 倍，比最差的 FastRoute 4.1 加速了 10 倍。

表 4-4 与三种布线算法的布线迭代时间结果比较

Name	Ours	SPRoute	FastRoute 4.1	NCTU-gr 2.0 Fast	NCTU-gr 2.0 Regular
adaptec1	11.384	13.859	71.458	11.080	46.450
adaptec2	1.286	2.532	12.054	6.220	8.180
adaptec3	8.429	10.559	49.599	12.080	52.930
adaptec4	0.724	0.778	2.694	6.530	3.380
adaptec5	18.067	24.349	128.574	37.060	182.170
bigblue1	16.502	20.016	90.635	24.480	105.630
bigblue2	78.612	97.160	262.050	27.560	79.860
bigblue3	3.996	4.124	19.569	12.460	27.190
bigblue4	360.151	1423.012	12125.000	51.080	115.200
newblue1	35.435	109.235	1321.241	28.190	63.240
newblue2	0.310	0.398	1.424	4.620	1.350
newblue3	140.226	85.081	386.370	109.920	13092.400
newblue4	231.217	853.722	2642.250	31.490	74.080
newblue5	23.345	35.376	175.406	45.330	99.430
newblue6	25.636	38.676	201.985	30.940	94.220
newblue7	2739.068	7694.989	26388.037	203.940	1017.920
avg	1	1.800	10.053	2.703	9.634

从与开源算法对比的实验中可以看出，本文提出的拥塞感知的全局布线算法在布线拥塞溢出的控制上较为优秀，与第一名在测试案例上的表现互有胜负，而在布线效率上十分优异，远超其他所有算法。因此，本文的算法不仅能够有效降低布线拥塞，生成高质量的解来指导详细布线，还极大地提升了布线效率。该算法能够准确估计布线拥塞情况，作为布线拥塞的估计工具为可布线性布局提供指导。

4.4 本章小结

本章先介绍了实验中使用的基准测试案例，随后对本文提出的拥塞感知的全局布线算法进行了有效性、拥塞解决能力和可伸缩性进行了验证，最后在该测试案例上与学术上多个优秀的开源学术布线算法进行了比较。

5 总结与展望

5.1 总结

在拥塞感知的全局布线中，成百上千万的网络需要在很小的矩形区域内有限的布线资源下连接，连线都需要占用线宽并满足线间距要求。因此不仅会面临网络过多而导致的网络冲突、拥塞严重的问题，也需要面临网络线长、通孔过多导致芯片性能、可制造性变差的问题，以及布线时间过长而增加设计周期。因此如何研究出能降低布线拥塞以及提升全局布线效率的算法具有重要的意义。

在全局布线算法研究中，都会不同程度地使用迷宫布线算法来在更大的解空间中寻找解，因此对于多源多目标的迷宫布线算法优化在布线中具有重要的价值。在全局布线中，存在撕裂重布线和整数线性规划布线两种布线迭代算法框架，设计出合适的迭代算法框架不仅能提升布线质量也能提升布线效率。由于全局布线算法发展目前没有重大的突破，所以当前很多的工作也落到了如何设计出高质量的并行算法在不影响布线拥塞的情况下来提升布线效率，对各个网络进行布线，网络间的路径都会存在不同程度的交叉而互相影响布线质量，因此需要合适的布线算法来调度布线网络，解决网络间的布线冲突。对此本文做出了以下工作：

（1）对多源多目标的迷宫布线算法进行了研究，从而设计了两阶段的双路径迷宫布线算法来降低布线搜索空间提升布线效率。虽然在快速布线拥塞估计具有较大的意义，但是该算法具有不确定性，不利于程序调试也不适合工业应用。于是在此基础上，本文又提出了确定性的双路径迷宫布线，在能够满足确定性要求的同时保证了布线质量，在算法描述之后证明算法的有效性。

（2）通过对弱一致性并行布线算法的研究，提出了新的网络级别的布线算法，不仅能在相同数量核心的机器上展现出更快的布线效率，更能在多核的设备中提高算法的整体收敛性能。

（3）提出了动态布线调度算法，对布线网络迭代不同阶段进行同步，序列化撕裂网络阶段，并对布线和网络拓扑结构更新部分进行了并行化，在利用网络搜索空间相斥的布线区域时，来解决网络间冲突的布线路径，避免拥塞无法收敛。最后再与迷宫布线相结合，进一步提升布线效率，减少布线冲突。

（4）将网络级别布线和动态布线相结合，设计了两阶段的布线框架，该算法在第一阶段能对布线资源充足区域的网络进行快速布线，然后通过设计的并

行度下降算法来控制并行度，在满足指定条件时切换到第二阶段执行动态布线调度算法，来对布线拥塞区域的网络进行布线。通过该布线框架不仅能很好地控制布线拥塞，同时还提升了布线效率。

5.2 展望

随着图形处理单元性能的提升和神经网络模型的发展，各行各业都在使用深度学习来解决问题。但是在全局布线领域仍然是深度学习难以克服的问题，目前也只能运用深度学习来进行简单地布线拥塞估计，而难以实现全局布线的功能。原因有二：其一是大规模布线单元数量带来的指数级别的布线解空间，其二是没有大量的集成电路设计样本作为训练集，而工业中一款芯片的设计与完成周期能达到一两年。由于全局布线问题是经典的组合优化问题，未来可以针对布线中各个部分算法，探索出能使用神经网络解决的子问题。此外，由于图形处理单元强大的性能，可以使用其在布线中加速一些传统算法，以此获得更快的求解速度。

参考文献

- [1] 董晨,陈震亦,尚艳艳.粒子群优化的集成电路多端点线网全局布线方法[J].微电子学与计算机,2013,30(11):171-176.DOI:10.19304/j.cnki.issn1000-7180.2013.11.041.
- [2] Chu C , Wong Y C . FLUTE: Fast Lookup Table Based Rectilinear Steiner Minimal Tree Algorithm for VLSI Design[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2008, 27(1):p.70-83.
- [3] Pan M, Chu C. FastRoute: A step to integrate global routing into placement[C]//Proceedings of the 2006 IEEE/ACM international conference on Computer-aided design. 2006: 464-471.
- [4] Neogi P P G. Novel Approach for Steiner Nodes Filtration in VLSI Global Routing[C]//2019 International Conference on Intelligent Computing and Remote Sensing (ICICRS). IEEE, 2019: 1-6.
- [5] Latha N R, Prasad G R. Wirelength and memory optimized rectilinear Steiner minimum tree routing[C]//2017 2nd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT). IEEE, 2017: 1493-1497.
- [6] Chen G, Young E F Y. Salt: provably good routing topology by a novel steiner shallow-light tree algorithm[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2019, 39(6): 1217-1230.
- [7] Moffitt M D. MaizeRouter: Engineering an effective global router[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2008, 27(11): 2017-2026.
- [8] Albrecht, C. Global routing by new approximation algorithms for multicommodity flow[J]. IEEE Trans on Cad/icas, 2001, 20(5):622-632.
- [9] Kothari A P. Parallelization of Pattern Routing Algorithms in Chip Design[J]. 2020.
- [10] Cho M , Pan D Z . BoxRouter: A New Global Router Based on Box Expansion and Progressive ILP[J]. IEEE Press, 2007.
- [11] Shojaei H , Davoodi A , Linderorth J T . Congestion analysis for global routing via integer programming[C]// 2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD). IEEE, 2011.
- [12] Shojaei H , Davoodi A , Linderorth J . Planning for local net congestion in global routing[C]// Proceedings of the 2013 ACM international symposium on International symposium on physical design. ACM, 2013.
- [13] Fontana T A, Aghaeekiasaraee E, Netto R, et al. ILP-based global routing optimization with cell movements[C]//2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI). IEEE, 2021: 25-30.

-
- [14] Liu W H , Kao W C , Li Y L , et al. NCTU-GR 2.0: Multithreaded Collision-Aware Global Routing with Bounded-Length Maze Routing[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2013, 32(5):709-722.
- [15] Chang Y J, Lee Y T, Wang T C. NTHU-Route 2.0: A fast and stable global router[C]//2008 IEEE/ACM International Conference on Computer-Aided Design. IEEE, 2008: 338-343.
- [16] Young E . CUGR: detailed-routability-driven 3D global routing with probabilistic resource model[C]// 2020 57th ACM/IEEE Design Automation Conference (DAC). ACM, 2020.
- [17] Pan M , Chu C . FastRoute 2.0 : A high-quality and efficient global router[J]. Proc. Asia and South Pacific Design Automation Conf. Jan. 2007, 2007:250-255.
- [18] Liu W H , Li Y L , Koh C K . A fast maze-free routing congestion estimator with hybrid unilateral monotonic routing[C]// Computer-Aided Design (ICCAD), 2012 IEEE/ACM International Conference on. ACM, 2012.
- [19] Yue X , Chu C . MGR: Multi-level global router[C]// 2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), San Jose, California, USA, November 7-10, 2011. ACM, 2011.
- [20] Cho M, Lu K, Yuan K, et al. BoxRouter 2.0: Architecture and implementation of a hybrid and robust global router[C]//2007 IEEE/ACM International Conference on Computer-Aided Design. IEEE, 2007: 503-508.
- [21] Cao Z, Jing T, Xiong J, et al. DpRouter: A fast and accurate dynamic-pattern-based global routing algorithm[C]//2007 Asia and South Pacific Design Automation Conference. IEEE, 2007: 256-261.
- [22] Chen J W, Di Z X, Chen J J, et al. An Enhanced Heuristic Layer Assignment Method in Global Routing[C]//2020 IEEE 15th International Conference on Solid-State & Integrated Circuit Technology (ICSICT). IEEE, 2020: 1-3.
- [23] Gao J R, Wu P C, Wang T C. A new global router for modern designs[C]//2008 Asia and South Pacific Design Automation Conference. IEEE, 2008: 232-237.
- [24] Roy J A , Markov I L . High-Performance Routing at the Nanometer Scale[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2008, 27(6):1066-1077.
- [25] Kastner R , Bozorgzadeh E , Sarrafzadeh M . Pattern routing: use and theory for increasing predictability and avoiding coupling[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2001, 21(7):777-790.
- [26] Xu Y , Zhang Y , Chu C . FastRoute 4.0: Global router with efficient via minimization[C]// Proceedings of the 14th Asia South Pacific Design Automation Conference, ASP-DAC 2009, Yokohama, Japan, January 19-22, 2009. 2009.
-

-
- [27] Lee, C. Y . An Algorithm for Path Connections and Its Applications[J]. Ire Trans on Electronic Computers, 1961, 10(3):346-365.
 - [28] Johann M, Reis R. Net by net routing with a new path search algorithm[C]//Proceedings 13th Symposium on Integrated Circuits and Systems Design (Cat. No. PR00843). IEEE, 2000: 144-149.
 - [29] Prim R C. Shortest connection networks and some generalizations[J]. The Bell System Technical Journal, 1957, 36(6): 1389-1401.
 - [30] Kruskal J B. On the shortest spanning subtree of a graph and the traveling salesman problem[J]. Proceedings of the American Mathematical society, 1956, 7(1): 48-50.
 - [31] Kuh E S , Ohtsuki T . Recent advances in VLSI layout[J]. Proceedings of the IEEE, 1990, 78(2):237-263.
 - [32] Hadsell R T, Madden P H. Improved global routing through congestion estimation[C]//Proceedings 2003. Design Automation Conference (IEEE Cat. No. 03CH37451). IEEE, 2003: 28-31.
 - [33] Cao Z , Jing T T , Xiong J , et al. Fashion: A Fast and Accurate Solution to Global Routing Problem[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2008, 27(4):726-737.
 - [34] Liu S, Pu Y, Liao P, et al. Fastgr: Global routing on cpu-gpu with heterogeneous task graph scheduler[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022.
 - [35] Lin S, Liu J, Young E F Y, et al. Gamer: Gpu accelerated maze routing[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022.
 - [36] Jiang Y J, Fang S Y. COALA: co ncurently a ssigning wire segments to la yers for 2D global routing[C]//Proceedings of the 39th International Conference on Computer-Aided Design. 2020: 1-8.
 - [37] McMurchie L, Ebeling C. PathFinder: A negotiation-based performance-driven router for FPGAs[C]//Proceedings of the 1995 ACM third international symposium on Field-programmable gate arrays. 1995: 111-117.
 - [38] Wei X, Tang W C, Diao Y, et al. ECO timing optimization with negotiation-based re-routing and logic re-structuring using spare cells[C]//17th Asia and South Pacific Design Automation Conference. IEEE, 2012: 511-516.
 - [39] Chen H Y, Hsu C H, Chang Y W. High-performance global routing with fast overflow reduction[C]//2009 Asia and South Pacific Design Automation Conference. IEEE, 2009: 582-587.
 - [40] He J, Burtcher M, Manohar R, et al. Sproute: A scalable parallel negotiation-based global

-
- router[C]//2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD). IEEE, 2019: 1-8.
- [41] Wu T H, Davoodi A, Linderoth J T. GRIP: Scalable 3D global routing using integer programming[C]//Proceedings of the 46th Annual Design Automation Conference. 2009: 320-325.
- [42] Wu T H, Davoodi A, Linderoth J T. A parallel integer programming approach to global routing[C]//Proceedings of the 47th Design Automation Conference. 2010: 194-199.
- [43] Shi D, Davoodi A, Linderoth J. A procedure for improving the distribution of congestion in global routing[C]//2016 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2016: 249-252.
- [44] Torabi M, Zhang L. Electromigration and Parasitic-Aware ILP Router for Analog and RF Circuits[C]//2018 IEEE International Symposium on Circuits and Systems (ISCAS). IEEE, 2018: 1-5.
- [45] Torabi M, Zhang L. Electromigration-and parasitic-aware ILP-based analog router[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2018, 26(10): 1854-1867.
- [46] Lee T H, Wang T C. Congestion-constrained layer assignment for via minimization in global routing[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2008, 27(9): 1643-1656.
- [47] Gao J R, Wu P C, Wang T C. A new global router for modern designs[C]//2008 Asia and South Pacific Design Automation Conference. IEEE, 2008: 232-237.
- [48] Lin S, Wong M D F. Superfast Full-Scale CPU-Accelerated Global Routing[C]//Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design. 2022: 1-8.
- [49] Pan M, Xu Y, Zhang Y, et al. FastRoute: An efficient and high-quality global router[J]. VLSI Design, 2012, 2012.
- [50] Zou P, Cai Z, Lin Z, et al. Incremental 3D Global Routing Considering Cell Movement and Complex Routing Constraints[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022.